Technische Hochschule **Rosenheim**

Fakultät für Informatik

Wirtschaftsinformatik

Konzeption und Entwicklung eines KI-gestützten Chatbots zur automatischen Beantwortung von Supportanfragen unter Verwendung interner Datenquellen

Bachelor Thesis

von

Atakan Sentürk

Datum der Abgabe: tt.mm.jjjj
Erstprüfer: Prof. Dr. Marcel Tilly
Zweitprüfer: Prof. Dr. Gerd Beneken

Eigenständigkeitserklärung / Declaration of Originality

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken (dazu zählen auch Internetquellen) entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht.

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Rosenheim, den tt.mm.jjjj

Vor- und Zuname

Kurzfassung

text

Schlagworte:

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis

Abkürzungen

1 Einführung

Durch die steigende Popularität von Chatbots wie ChatGPT von OpenAI und Claude von Anthropic in den letzten Jahren, wie man in **??** sehen kann, werden die Fortschritte, die im Bereich der künstlichen Intelligenz erreicht werden, immer bekannter und der Nutzen den KI Applikationen im Leben von Menschen haben wird dadurch immer erweitert.

Chatbots sind inzwischen in der Lage, diverse Aufgaben im Bereich der Kundekommunikation zu übernehmen, weshalb Unternehmen den ersten Schritt ihres Kundensupports durch diese ersetzen. Dies wird bei manchen Unternehmen über ein Chat Interface auf ihrer Website eingebaut. Über dieses Interface können die Nutzer fragen stellen und Probleme lösen lassen, falls keine Lösung gefunden werden kann, werden sie dann an den Kundensupport weitergeleitet **??**.

1.1 Motivation

Die Motivation für diese Bachelorarbeit entspringt der Erkenntnis, dass trotz konstanter Forstschritte im Bereich der Large Language Models und Chatbots, weiterer Forschungs- und Entwicklungsbedarf besteht.

Die Inhalte dieser Bachelorarbeit wurden zusammen mit der Techdivision GmbH erarbeitet. Die Techdivision hat firmenintern Support Chats, in denen die Mitarbeiter der Firma, bei technischen Problemen Fragen stellen können und diese vom IT-Support Team beantwortet werden.

Einige dieser Fragen sind wiederkehrende Fragen, die mithilfe der firmeninternen Dokumentation gelöst werden könnten, wie die Frage, wieso man sich nicht bei Bitwarden mit dem Master Passwort einloggen kann. Um die Ressourcen des IT-Teams zu schonen und die Probleme der Fachbereiche schneller zu lösen, soll ein Chatbot erstellt werden, durch Zugriff auf Daten der Firma Fragen innerhalb des Support Chats beatnwrotet.

Diese Arbeit zielt darauf ab, einen Beitrag zur Überbrückung der Lücke zwischen theoretischen Konzepten und praktischer Anwendung zu leisten. Durch die konzeptionelle Entwicklung eines Chatbots und die anschließende prototypische Implementierung wird ein Ansatz verfolgt, der es ermöglicht, verschiedene LLMs im Kontext realer Anwendungsszenarien zu evaluieren.

Der Vergleich von vier unterschiedlichen LLMs innerhalb des entwickelten Prototyps bietet die Gelegenheit, die Leistungsfähigkeit, Effizienz und Anwendbarkeit dieser Modelle zu untersuchen. Diese vergleichende Analyse soll zur Ermittlung des am besten geeigneten Modells für diesen spezifischen Anwendungsfall beitragen.

Darüber hinaus adressiert diese Arbeit die wachsende Nachfrage nach praxisorientierten Lösungen im Bereich der KI-gestützten Kommunikation. In einer Zeit, in der Unternehmen und Organisationen zunehmend auf KI-Systeme setzen, um Effizienz zu steigern und Kundenerlebnisse zu verbessern, liefert diese Arbeit konkrete Erkenntnisse für die Entwicklung und den Einsatz moderner Chatbots.

1.2 Zielsetzung

Das Ziel der Arbeit besteht darin, einen Chatbot zu konzipieren und einen Prototyp zu entwickeln, der mit Hilfe der Daten der internen Datenquellen der Techdivision Support leistet und Fragen beantwortet. Der Chatbot soll innerhalb von Google Chat integriert werden. Dabei ist es wichtig, dass der Chatbot, statt in Einzelchats, in einer Chat Gruppe läuft. Diese beinhaltet neben den Support Suchenden, ebenfalls Mitarbeiter des jeweiligen Services. Auf diese Weise kann, falls, notwendig, extern eingegriffen werden. Außerdem sollte der Chatbot in der Lage sein zu überprüfen, wie viele der aktuellen Supportanfragen er eigenständig beantworten oder lösen kann. Falls der Chatbot keine Antwort auf eine Anfrage haben sollte, soll er in den angebundenen Datenquellen nachschlagen können, um mit dem Wissen seine Wissensbasis zu erweitern und die Fragen entsprechend zu beantworten.

Hierfür wird im Rahmen dieser Arbeit ein Konzept für einen Chatbot erstellt, welches prototypisch implementiert wird. Dazu werden über die Implementierung vier verschiedene LLMs verwendet und miteinander verglichen, um das beste für diesen Anwendungsfall zu ermitteln. Der Vergleich wird in zwei Stufen durchgeführt. In der ersten Stufe werden die LLMs über das Framework Ragas verglichen. Im zweiten Schritt werden die zwei besten LLMs aus dem Ragas Vergleich, über einen Fragebogen Vergleich von Mitarbeitern der TD bewertet und miteinander verglichen.

2 Grundlagen

2.1 Chatbots

Ein Chatbot ist ein Dialogsystem, das den Dialog zwischen einem Menschen und einem technischen System erlaubt [**?**]. Von diesen gibt es verschiedene Arten. Diese unterscheiden sich in der Art und Weise, wie sie auf die Eingaben des Nutzers antworten. Chatbots, wie OpenAIs ChatGPT und Anthropics Claude, haben einen Bereich zur Texteingabe und einen Bereich, in dem sie ihre Ausgabe darstellen können. Mit diesen kann man heutzutage über die natürliche menschliche Sprache kommunizieren und sie versuchen auf dieselbe Art und Weise zu antworten.

Die Generation des Textes kann in drei verschiedene Kategorien untergeordnet werden: abfragebasiert, generationsbasiert und regelbasiert [**?**]. Abfragebasierte Chatbots haben eine Wissensdatenbank in der sie nach dem am besten zur Eingabe passenden Eintrag suchen. Beim generationsbasierten Ansatz generieren sie ihre Antworten selbst, wie ein paar dieser Modelle ihre Antworten generieren wird in **??** erklärt. Regelbasierte Chatbos antworten nach vorher in ihrem Skript bestimmten Regeln. Hierbei vergleichen sie die Eingabe mit ihren vorher definierten Regeln und transformieren die Eingabe, falls möglich, in eine vorher definierte Satzstruktur. [**?**] Im nächsten Schritt wird darauf eingegangen, wie sich Chatbots in dieser Hinsicht mit der Zeit verändert haben.

2.1.1 Turing Test

Im Jahr 1950 hat Alan Turing den Turing Test vorgeschlagen, bei dem es darum geht zu erkennen, ob eine Maschine das gleiche Denkvermögen hat wie ein Mensch. Bei diesem Test

werden einem Tester zwei Gesprächspartner vorgelegt, von denen einer ein Mensch und der andere eine Maschine ist. Der Tester führt dann ein Gespräch mit beiden Gesprächspartnern und befragt diese. Nach der Befragung wird überprüft, ob der Tester in der Lage ist, zu sagen, welcher der Partner eine Maschine ist. Falls der Tester nicht in der Lage ist zu erkennen, welcher der Partner die Maschine ist, gilt der Turing Test als bestanden und die Maschine ist laut Turing intelligent [**?**].

2.1.2 ELIZA

Die Geschichte der Chatbots beginnt in den 1960er Jahren mit dem ersten aus dem Jahr 1966. Damals waren die Chatbots regel basiert. Der Name des Chatbots war ELIZA und er wurde von Joseph Weizenbaum am MIT entwickelt. Weizenbaum hat fünf Grundprobleme aufgestellt, welche er für die Entwicklung beachten musste . Die Probleme waren:

1. Die Identifikation von Stichwörtern
2. Das Finden des minimalen Kontexts
3. Die Auswahl der geeigneten Transformation
4. Generation von Antworten bei Abwesenheit von Stichworten
5. Bearbeitung der Eliza Skripte

[**?**].

Die Regeln, die ELIZA beachten muss, wurden ihr über Skripte gegeben. Sie hat versucht die menschliche Sprache zu imitieren, indem sie den eingegebenen Text nach Stichwörtern durchsucht. Stichwörter sind Wörter, denen besonders hohe Wichtigkeit laut dem ELIZA Skript gegeben wurde. Jedes Stichwort hatte noch einen RANK, welcher die Wichtigkeit des jeweiligen Stichworts im Vergleich zu anderen Stichworten festgelegt hat. Nachdem die Stichwörter entdeckt wurden, betrachtet ELIZA die Transformations Regeln, die in ihrem Skript bestimmt wurden. Die Transformations Regeln bestehen aus einer Zerlegungs und einer Zusammensetzungs-Phase. Es gibt verschiedene Regeln für beide Phasen. Anhand der Syntax des Satzes wird identifiziert, welche Regel für die Zerlegung in Frage kommt und anhand dieser wird dann die Zusammensetzung ausgemacht. Dieser Zusammengesetzte Satz wird im Anschluss ausgegeben Im Fall, dass kein Stichwort gefunden wurde, wird entweder eine inhaltslose Antwort oder eine frühere Antwort zurück gegeben. ELIZA verhält sich wie ein Psychotherapeut. Das heißt, der Nutzer beginnt das Gespräch und der Chatbot greift Punkte aus den Eingaben des Nutzers auf, ohne großen Zusammenhang mit Themen außerhalb dieses Gesprächs und ihrer Regeln. Daraus lässt sich schließen, dass ELIZA keinen Kontext kennt, sondern, je nach seinem aktuellen Skript auf den aktuellen Satz antwortet [**?**].

2.1.3 Parry

Der nächste Chatbot, Parry, wurde an der Stanford University entwickelt. Während Eliza einen Psychotherapeuten imitiert hat, hat Parry einen Patienten mit paranoider Schizophrenie imitiert. Dieser Bot war im Jahr 1979 Teil eines Experiments, der einem Turing Test (siehe **??**) gleicht. Hier wurden verschieden Psychotherapeuten vor Bildschirme gesetzt und mussten erraten, ob die gegenüber gestellten Aussagen von einem echten Schizophrenen Patienten kommen oder von einem Chatbot. Teil des Experiments waren 5 Psychiater, die jeweils 2 Patienten zur Beurteilung bekommen haben.

Von den 5 Teilnehmern lag nur einer bei beiden richtig. Einer lag bei beiden falsch, ein weiterer dachte beide Patienten sind echte Patienten und die letzten zwei dachten, dass beide Patienten Chatbots sind [**?**].

Das Problem hierbei war jedoch, dass der Pool an Probanden mit 5 Psychiatern sehr klein war. Dazu kommt noch der Fakt, dass Patienten mit Schizophrenie ein gewisses Level an Unverständlichkeit in ihren Konversationen haben, welches die Beurteilung, ob es generell ein guter Chatbot ist erschwert. Heute wird Parry eher als ein schwacher Chatbot gesehen, da er nicht von Konversationen lernen kann, keinen richtigen Kontext hat und sehr langsam ist. Natürlich ist ein Teil davon abhängig von den technischen Limitationen die es damals gab, aber es ist dennoch wichtig zu beachten [**?**].

2.1.4 ALICE

1995 wurde der Internet Chatbot ALICE veröffentlicht. Dies war der erste Chatbot der im Internet zu finden war und er wurde als Open Source Software veröffentlicht. Im Laufe der Jahre wurde das Projekt mehrmals mit verschiedenen Preise ausgezeichnet, wie beispielsweise dem Loebner Prize, mit dem das Programm ausgezeichnet wird, welches von der Jury als am menschenähnlichsten bewertet wird **??**. Dieser Chatbot wurde von Weizenbaums ELIZA inspiriert und war selbst die Inspiration für den Film "Her", in der die Beziehung zwischen Menschen und Maschinen betrachtet wird [**?**]. Alice hatte eine viel größere Sammlung an Stichwörtern, auf die sie reagieren konnte als ELIZA, aber sie hat auf die selbe Art und Weise Stichwörter in den Eingaben des Nutzers gesucht und dann je nach ihren Datenbankeinträgen darauf geantwortet [**?**, **?**].

2.1.5 Siri, Google Assistant, Alexa

Nach diesen anfänglichen Versuchen, welche vorallem im Bereich der Psychologie entwickelt wurden und die regelbasiert waren, wurden immer weiter Fortschritte gemacht.

Eine große Neuerung im Jahr 2010 war Siri, ein persönlicher Assistent auf dem eigenen Handy. Siri, welches nur auf Apple Geräten zur Vefügung steht, hilft den Nutzern, welche über Text oder Sprachbefehle mit dem Assistenten kommunizieren können, mit verschiedenen Aufgaben. Unter anderem können die Nutzer mittels Siri Erinnerungen setzen, Nachrichten senden und Anrufe tätigen. [**?**]

Sprachassistenten wie Siri gibt es auch auf anderen Plattformen, wie auf Samsung Galaxy Geräten mit Galaxy AI und verschiedenen Android Geräten mit Google Assistant. Diese Assistenten haben ähnliche Funktionen wie Siri und versuchen dem Nutzer zu helfen[**?**, **?**].

SmartHome Geräte, wie Smarte Lampen und Steckdosen lassen sich mit Hilfe von diesen Sprachassisten kontrollieren.

Manche Sprachassistenten lassen sich über extra Geräte, die für diesen Nutzen hergestellte wurden. Ein Beispiel hierfür ist Amazon Alexa, auf dem Echo Dot. Der Echo Dot ist ein kleiner Lautsprecher, welcher mit dem Internet verbunden ist. Durch diesen kann man mit Alexa kommunizieren. Diese Sprachassisten werden immer weiter entwickelt und bekommen dadurch neue Funktionen, mehr Sprachen oder die Fähigkeit die menschliche Sprache besser zu interpretieren.

2.1.6 ChatGPT, Claude, Gemini

Wie in **??** bereits gezeigt, steigen aktuell die Anfragen für Chatbots wie ChatGPT und Claude. Diese Chatbots basieren auf Large Language Models, welche in **??** genauer erklärt werden. Nutzer können teils über Apps und teils über die Website, des jeweiligen Services, mit diesen Bots kommunizieren.

Den Chatbots werden fragen stellen, welche vom jeweiligen System beantwortet werden. Sie können nicht nur in Textform antworten, sondern auch durch Bild oder Audiogenerierung, wie bei ChatGPTs DALL·E Integration. Diese Chatbots sind nicht nur in der Lage Fragen zu beantworten, sondern sie können auch Code, einen Essay oder ein Film Script generieren.

2.2 Künstliche Intelligenz

Künstliche Intelligenz ist ein Teil der Informatik der sich damit beschäftigt, Aufgaben, die früher menschliche Intelligenz benötigt haben, von Maschinen bzw. Computern übernehmen zu lassen. Zu den Aufgaben gehören unter anderem:

- Erkennen von Mustern
- Treffen von Entscheidungen
- Verstehen von Sprache

Durch Fortschritte in der künstlichen Intelligenz konnten Forstschritte in all diesen Aufgabenbereichen erreicht werden. Die verschiedenen Aufgaben werden in verschiedenen Teilbereichen der künstlichen Intelligenz behandelt. Ein paar der Teilbereiche sind:

- Computer Vision
- Natural Language Processing
- Robotik

Nun ist die Frage wie Intelligenz definiert werden kann. Das ist keine einfache Aufgabe und nicht der Schwerpunkt dieser Arbeit. Im Kontext dieser Arbeit wird das Buch “Knowledge Science – Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten” von Lanquillong und Schacht und die Definition von Intelligenz daraus betrachtet. Laut Lanquillon und Schacht ist eine Möglichkeit ein paar Eigenschaften anzuschauen, welche an menschliches Verhalten angelehnt sind und davon ausgehend künstliche Intelligenz definieren. Diese Liste ist offen und nicht vollständig. Die Eigenschaften die im Buch genannt werden sind:

- Wahrnehmung
- Schlussfolgerung
- Handeln
- Planung und zielgerichtete Problemlösung
- Kommunikation
- Anpassungsfähigkeit und Lernen

- Autonomie
- Kreativität
- Reflexion und Bewusstheit
- Ästhetik
- Organisation

Die meisten Menschen weisen alle diese Eigenschaften auf, aber es gibt noch keine künstliche Intelligenz, die alle davon aufweisen kann. Da es noch kein System gibt, welches in diese Kategorie fällt, aber es bereits Systeme gibt, die ein paar dieser Eigenschaften aufweisen und dazu noch Probleme in eng definierten Problembereichen lösen können. Ist eine weitere Unterscheidung innerhalb des Bereichs der künstlichen Intelligenz nötig. Daraus formen sich die zwei Begriffe schwache künstliche Intelligenz und starke künstliche Intelligenz [**?**, S. 5-7].

2.2.1 Schwache künstliche Intelligenz

Die schwache künstliche Intelligenz ist ein System, welches spezifische Probleme innerhalb einer bestimmten Domäne lösen kann. Diese Art der künstlichen Intelligenz wird darauf trainiert, Probleme innerhalb eines eng begrenzten Bereichs zu lösen, indem sie speziell auf diesen Bereich trainiert wird und im Laufe dessen dann die Regeln dieses Bereichs kennen lernt. Nachdem es trainiert wurde, kann die schwache künstliche Intelligenz durch die beim Training erlernten Regeln und Muster, Probleme innerhalb der Domäne in der es trainiert wurde lösen. Es kann dieses "Wissenäber nicht auf andere Problembereiche übertragen und dadurch Probleme in dieser neuen Domäne lösen.

Ein Beispiel für eine schwache künstliche Intelligenz wäre ein System, welches darauf trainiert wurde Texte von einer Sprache in einer andere zu übersetzen. Das System kann vielleicht innerhalb dieser Domäne die Aufgabe sehr gut erfüllen, was heißt, dass sie die Texte mit hoher Genauigkeit in die Zielsprache übersetzen kann, ohne den Verlust von Informationen, aber egal wie gut es in dieser Domäne ist, bleibt das System auf diese Domäne beschränkt. Das heißt, es kann das erlente Wissen, nicht auf unverwandte Aufgaben übertragen. [**?**].

2.2.2 Starke künstliche Intelligenz

Starke künstliche Intelligenz oder Allgemeine künstliche Intelligenz, wäre ein System, welches nicht nur die in **??** genannte Liste an Eigenschaften aufweisen könnte, sondern auch in der Lage wäre, Probleme zu lösen, ohne explizit für diese trainiert worden zu sein. Die starke künstliche Intelligenz, wäre in der Lage ihre erlentes Wissen zu verallgemeinern und es dadurch, ähnlich wie ein Mensch, auf neue Anwendungsbereiche anzuwenden. Darüber hinaus, wäre eine starke künstliche Intelligenz ebenfalls in der Lage sich selbst weiterzuentwickeln und zu lernen, ohne das menschliches Eingreifen erforderlich wäre. Diese Fähigkeit würde bedeuten, dass starke künstliche Intelligenz, durch diese Selbstverberbesserungsfähikeit in der Lage wäre ihr eigenes System und ihre eigenene Algorithmen zu optimieren und somit irgendwann die menschliche Intelligenz zu übertreffen [**?**].

Solch ein System würde nicht nur durch seine Problemlösungsfähigkeit ausgezeichnet sein, sonder ebenfalls durch Dinge wie kreatives Denken und emotionale Intelligenz. Dadurch wäre sie in fast allen Bereichen einsetzbar, in denen diese Eigenschaften gefragt sind [**?**].

Mit dem Aufstieg von künstlichen Intelligenzen im Auge der Öffentlichkeit, zeigen sich einige Menschen besorgt, was passieren würde, wenn solch ein System erschaffen werden würde. Es ist jedoch zu betonen, dass ein solches System noch nicht existiert und es ist davon auszugehen, dass dies auch in naher Zukunft nicht der Fall sein wird [**?**].

2.2.3 Natural Language Processing

Natural Language Processing oder natürliche Spracherkennung, ist ein Teil des Machine Learnings der sich auf die Arbeit mit menschlicher, natürlicher Sprache fokussiert. Das Ziel von Natural Language Processing ist es, Computern die Möglichkeit zu geben, menschliche Sprache zu verstehen und zu verarbeiten, um mit den Menschen kommunizieren zu können. Dies schließt Kommunikation in Form von Audio als auch über Text mit ein [**?**].

Natural Language Processing ermöglicht es Computern, durch Kombination verschiedener Machine Learning Verfahren und anderer linguistischer Verfahren menschliche Sprache zu verstehen und diese auch wiederzugeben. Diese Systeme können, Texte zusammenzufassen, sie analysieren, ihre Bedeutung zu verstehen und mit Hilfe dieser Fähigkeiten dann sinnvolle und passende Antworten zu generieren. Anwendungen die Natural Language Processing nutzen sind vielfältig, Systeme wie Chatbots und Übersetzungssoftware implementieren auch NLP Verfahren.[**?**]

Da Sprache oft kontextabhängig ist und manche Sprachen, wie Japanisch komplette Satzteile weg lassen, wenn diese durch den Kontext erkenntlich sein sollten, ist es ein wichtiger Teil des Natural Language Processing, in der Lage zu sein kontextuelle Informationen zu berücksichtigen. Dadurch kann das System Kontext verstehen und richtig interpretieren. Dies wird über verschiedene Methoden erreicht, wie Tokenisierung, Parsing und Named Entity Recognition. Tokenisierung ist ein Verfahren, welches LLMs nutzen. Dieses wird in **??** genauer erklärt.[**?**]

NLP stellt eine wichtige Schlüsselstelle in der Human-Computer Interaction oder zu deutsch Mensch-Computer-Interaktion dar. Sie hilft nicht nur dabei menschliche Kommunikation mit Computern einfacher und schneller zu machen, sondern gibt auch Menschen, die aufgrund von körperlichen Problemen ein Handy nicht ohne Probleme bedienen können, die Möglichkeit mit diesen Geräten zu interagieren[**?**].

2.2.4 Neuronale Netzwerke

Neuronale Netze sind ein Teil des Machine Learnings, welcher versucht sich stark an dem Aufbau eines biologischen Gehirns zu orientieren. Dementsprechend bestehen sie aus Tausenden oder sogar Millionen “Neuronen”, welche durch Knotenpunkte dargestellt werden. Diese “Neuronen” sind miteinander verbunden. Sie sind über verschiedene Reihen miteinander verbunden, wie in der **??** dargestellt.

Jeder Eingang an einem Neuron hat eine Gewichtung. Wenn die Gewichtung eines Eingangs 0 ist, dann heißt es, dass zwei Knoten keine Verbindung miteinander haben. Wenn die Gewichtung positiv ist, dann hat es eine bestärkende Wirkung und, falls die Gewichtung negativ ist, dann hat es eine hemmende Wirkung.

Jedes Neuron hat zudem einen sogenannten Bias. Durch den Bias hat das Neuron eine Grundaktivität, welche, ebenfalls während dem Training angepasst werden kann [**?**]. Das

aktuelle Neuron summiert dann alle gewichteten Werte und addiert den Bias. Wie in dieser Formel dargestellt:

$$\sum_i w_i x_i + b$$

Der summierte Wert wird durch die Aktivierungsfunktion transformiert und das Ergebnis wird dann dem nächsten Neuron in der nächsten Schicht weitergeleitet. Typische Aktivierungsfunktionen sind

- Sigmoid Function :
$$f(x) = \frac{1}{1 + e^{-x}}$$
- ReLu (Rectified Linear Unit):
$$f(x) = \max(0, x)$$
- Tanh:
$$f(x) = tanh(x)$$

[**?**] Bei der Eingabe Schicht kommt es auf die Trainingsdaten an, diese legen die Anzahl der Neuronen fest. Um genau zu sein, kommt es auf die Anzahl der Features an, die die Trainingsdaten beinhalten. Bei einem Neuronalen Netz, welches mit Daten aus einer normalen Tabelle trainiert wird, ist das beispielsweise die Anzahl der Spalten, bei einem bei einem Neuronalen Netz, welches mit Bildern trainiert werden soll, könnten das die einzelnen Pixel des jeweiligen Bildes sein [**?**].

Die Anzahl der Neuronen in der Ausgabe Schicht kommt darauf an, welche Art von Modell man betrachtet. Die zwei Arten die hier besprochen werden sind Regressoren und Klassifikatoren [**?**].

Es gibt zwei Arten von Regressoren. Regressoren sagen für einen Eingabewert einen kontinuierlichen Ausgabewert hervor [**?**, S. 100].

Beispiele hierfür sind:

- Gesichtserkennung
- Stimmerkennung
- Spamfilter in emails

Klassifikatoren sind dafür zuständig die Eingabe einer bestimmten Klasse zuzuordnen [**?**, S. 101]. Die Anzahl der Neuronen kommt auf die Anzahl der möglichen Klassen an **??**. Beispielsweise könnte man ein System betrachten, welches lernen soll, welche Art von Fahrzeug auf einem Bild dargestellt ist. In der **??** ist ein simpler Decision Tree dargestellt. Anhand dieser kann man sehen, welche Optionen es als Ausgaben gibt. In dieser Situation hätte die Ausgangs Schicht drei Neuronen, da es drei mögliche Ausgaben gibt.

2.2.5 RNN

Bevor auf die Transformer Architektur eingegangen werden kann, ist es wichtig, die vorangegangenen Architekturen zu verstehen. Dies ist wichtig, um zu verstehen, wie es zu den heutigen Architekturen gekommen ist und um zu erkennen, welche Vorteile diese bieten.

Bei der Weiterentwicklung von Neuronalen Netzen wurden Recurrent Neural Networks entwickelt. Bei diesen werden als Eingabe Sequenzen benutzt, die aufgeteilt werden. Die

einzelnen Teile werden dem Neuronalen Netz als Eingabe gegeben und während der Verarbeitung entsteht ein hidden State in dem RNN, welcher die Verarbeitung der nächsten Teile der Sequenzen beeinflusst [**?**].

Dadurch wird das Problem der Neuronalen Netze beseitigt, bei dem ein Neuronales Netz keine Erinnerungen hat und keinen Kontext hat und deshalb nur auf die aktuelle Eingabe reagieren kann.

2.2.6 LSTM

Ein Problem bei RNNs ist das Vanishing Gradient Problem, welcher im Zusammenhang steht, mit Backpropagation, welches ein Algorithmus ist, der zum Anpassen der Gewichte und des Biases von der Ausgabe zurückgehend bis zur Eingabe die Gradienten des Fehlers berechnet und dadurch die Leistung des Modells verbessert. Ein Problem hierbei ist, dass Gradienten die weit zurück liegen explodieren oder verschwinden können. [**?**]

Um dieses Problem zu lösen wurde Long Short Term Memory eingeführt. Dieser Ansatz ersetzt die rekurrenten Zellen des RNNs mit LSTM Zellen. Diese Zellen haben nicht nur einen hidden State, sondern auch einen Cell State, der für das Langzeitgedächtnis genutzt wird. Die LSTM Zelle hat drei Gatter, ein Eingabe Gatter, ein Ausgabe Gatter und ein Gatter, welches für das Vergessen von Informationen zuständig ist. [**?**]

Dieser Ansatz löst das Vanishing Gradient Problem und sorgt dafür, dass RNNs mit LSTM einen längeren Kontext sinnvoll verarbeiten als alleinstehende RNNs.

2.2.7 Encoder-Decoder Architektur

Diese Technologien haben sich damals vorallem im Kontext des Sequence-to-Sequence Learnings weiterentwickelt. Sequence-to-Sequence Learning oder kurz Seq2Seq Learning, bringt eine Sequenz variabler Länge in eine andere Sequenz variabler Länge.[**?**] Beispiele hierfür wären Text Zusammenfassungen und Übersetzungen. Die Architektur aus der sich der Transformer, der in dem Abschnitt **??** behandelt wird, entwickelt hat, ist die Encoder - Decoder Architektur.

Die Encoder-Decoder Architektur besteht aus zwei Rekurrenten Neuralen Netzen. Wie ein Neurales Netz funktioniert wurde bereits im Abschnitt **??** behandelt. Das besondere an einem Rekurrenten Neuralen Netz ist, dass dieses die vorherigen Aktivierungen der einzelnen Neuronen mit berücksichtigt. Dadurch, dass die vorangeganen Aktivierungen ebenfalls beachtet werden, können Kontexte erkannt werden und Sätze können sinnvoll übersetzt werden [**?**, **?**].

Hierbei erhählt der Encoder eine Eingabe Sequenz, welche dann Kontext Vektoren erstellt, welcher vom Decoder in eine Ausgabe Sequez umgewandelt wird. Das revolutionäre an dieser Technologie war damals, dass die Eingbae und Ausgabesequenzen variable Längen haben konnten. Der Kontext Vektor hatte jedoch eine feste Länge. Dieses Originale Encoder-Decoder Modell hatte trotz der Vorteile, die es geboten hat, jedoch einige Probleme:

1. Sequenzielles Verarbeiten: RNNs und LSTMs verarbeiten Sequenzen sequentiell. Dadurch, dass die Daten nich parallel verarbeitet werden, kommt es zu längeren Trainingszeiten.

2. Abhängigkeiten: In RNNs ist es schwierig langfristige Abhängigkeiten zu erfassen, aufgrund des Vanishing Gradient Problems, welches im Abschnitt **??** genauer erklärt wird.

3. Festes Kontextfenster: Da die meisten RNNs und LSTMs eine feste Kontextgröße haben, kann nur eine begrenzte Menge an Informationen als Kontext, zu einem bestimmte Zeipunkt, beachtet werden. Dadurch kann es zum Verlust von Informationen oder zur Verringerung der Genauigkeit der Antworten kommen. Besonders bei Sätzen die länger sind als die, in den Trainingsdaten, kann es hier zu Problemen kommen.

[**?**, **?**]

2.2.8 Attention Mechanismus

Die Idee auf die die Transformer aufbauen ist die des Attention Mechanism, welcher 2014 in der Arbeit “Neural Machine Translation by Jointly Learning to Align and Translate” von Bahdanau et al. vorgestellt wurde. Zu Beginn wurde der Mechanismus genutzt um die in **??** beschriebene Encode-Decoder Architektur zu verbessern, indem es vorallem auf das Problem der festen Vektorgröße des Kontextfensters eingeht [**?**].

Die Grundidee hierbei ist, dass jedes mal, wenn das System ein Wort übersetzt, es im zu übersetzenden Satz an den Stellen an denen sich die relevantesten Informationen sucht und mithilfe der Kontextvektoren der Wörter an diesen Stellen und der Kontextvektoren der Wörter die bereits übersetzt wurden, wird das nächste Wort generiert [**?**].

Der größte Unterschied zu originalen Encoder-Decoder Architektur ist dadurch, dass diese neue Architektur nicht versucht die gesamte Eingabesequenz in einen Kontextvektor umzuwandeln, sondern die Eingabesequenz in eine Sequenz von Vektoren umzuwandelt. Dadurch können die Modelle besser mit längeren Sätzen umgehen [**?**].

2.3 LLMs

Large Language Models sind eine Art der künstlichen Intelligenz, welche mithilfe probalistischer Methoden Text generiert. Heutzutage basieren Chatbots auf sogenannten Large Language Models.

Diese Large Language Models sind Machine Learning Modelle, die auf der Transformer Architektur (siehe **??**) aufbauen. Die Modelle werden mit einer sehr großen Menge an Daten trainiert und lernen anhand dieser ihre jeweilige Aufgabe. LLMs haben in verschiedenen Anwendungsgebieten einen nutzen. Heutzutage sind ein paar der bekanntesten Vertretetr Chatbots und Bildgeneratoren, wie Midjourney. [**?**].

2.3.1 Transformer

Die Transformer Architektur wurde 2017 von Vaswani et al im Paper Ättention is all you need"vorgestellt. Vor dieser Architektur waren die besten Modelle Encoder-Decoder Architekturen (siehe **??**), die mit dem Attention Mechanismus (siehe **??**) ergänzt wurden und die auf Recurrent oder Convolutional Neuronalen Netzen aufgebaut waren.

Der Originale Transformer, der von Vaswani et al vorgestellt wurde, bestand ebenfalls aus einem Encoder und einem Decoder. Jedoch wurden keine Recurrent oder Convolutional Neuronalen Netze mehr genutzt. Die großen Neuheiten in dieser Architektur war das Konzept der Self-Attention, Parallelisierbarkeit, "Multi-Head Attentionünd das weg lassen der recurrent oder Convolutional Neuronalen Netzen. [**?**]

Self-Attention

Während beim ursprünglichen Attention Mechanismus (siehe **??**) Beziehungen zwischen dem Encoder und dem Decoder modelliert wurden, ermöglicht Self-Attention es dem Modell, Beziehungen innerhalb einer einzelnen Sequenz zu erfassen.

Dies wird erreicht indem jedes Element einer Sequenz mit allen andere Elementen in Beziehung gesetzt wird. Dies hilft dabei den Kontext zu bestimmen.

Das heißt, durch die Self-Attention erkennt der Transfomer an welchen Position der Eingabe Sequenz er nachschauen muss um den aktuellen Vektor besser einbetten zu können.

Bei Self-Attention wird jeder Eingabe-Vektor durch drei Vektoren repräsentiert. Einem Query Vektor **Q**, einem Key Vektor **K** und einem Value Vektor **V**. Diese Vektoren werden erstellt indem der eingebette Eingabe Vektor **X** mit jeweils einer der Gewichts Matrizen W^Q, W^K, W^V multipliziert wird, welche während dem Training erstellt wurden.

$$Q = X\mathbf{W}^Q, K = X\mathbf{W}^K, V = X\mathbf{W}^V$$

Im nächstren Schritt werden die Skalarprodukte, die hier als Score bezeichnet werden, des Query Vektors des aktuell eingebetteten Vektors mit jeweils jedem Key Vektor der Token der aktuellen Eingabe berechnet.

$$Score_1 = Q_1 \cdot K_1$$
$$Score_2 = Q_1 \cdot K_2$$
$$...$$
$$Score_n = Q_1 \cdot K_n$$

Anschließend wird dieses Skalarprodukt durch die Wurzel der Dimensionen des Key Vektors d_k geteilt und auf das Ergebnis wird eine Softmax Funktion angewendet, die diese Werte normalisiert und dafür sorgt, dass sie summiert 1 ergeben.

$$ScaledScore = softmax(\frac{Score_1}{\sqrt{d_k}})$$

Als nächstes werden die Softmax Ergebnisse mit den Value Vektoren multipliziert und diese gewichteten Werte werden aufsummiert.

$$Attention = \sum_{i=1}^{n} ScaledScore_i V_i$$

Dies wird durch Matrix Berechnungen in einem Schritt erledigt mit der Formel:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Dies gibt uns den Attention Vektor für jeden Token.**[?, ?]**

Multi-Head Attention

Multi-Head Attention wurde ebenfalls mit dem Self-Attention Mechanismus in der Transformer Architektur eingeführt. Durch diesen soll dem Transformer die Fähigkeit gegeben werden sich auf verschiedene Positionen zu konzentrieren, statt nur auf eine **[?, ?]**.

Dieser Mechanismus funktioniert so, dass man statt nur einem Attention Head in der Attention Layer mehrere Attention Heads hat. Im originalen Transformer wird mit 8 Attention

Heads gearbeitet. jeder dieser Heads führt den Self-Attention Mechanismus aus, aber jeder hat unterschiedliche Gewichts Matrizen

$$W^Q, W^K, W^V$$

Jeder der Heads führt seine Berechnung mit unterschiedlichen Matrizen durch, was dazu führt, dass verschiedene Heads verschiedene Werte auf unterschiedliche Teile des Satzes legen können. Durch diesen Mechanismus kann es zu einem besseren Verständnis für Nuancen kommen [**?**, **?**].

Die finalen Attention Matrix z, die dem jeweiligen Token zugeschrieben wird, wird über eine weitere Matrix berechnet. Hierfür werden die Ergebnisse der Attention Heads $head_1, head_2, ..., head_n$ aneinander konkateniert und im Anschluss mit der Gewichts Matrix W_0 multipliziert [**?**, **?**].

$$z = Concat(head_1, head_2, ..., head_n)W_0$$

Sliding-Window Attention

Sliding Window Attention ist eine Weiterentwicklung der Self-Attention. Hierbei werden nicht alle anderen Position betrachtet, sondern nur eine bestimmte Menge an Positionen, die um den aktuellen Token liegen. Dies hat den Vorteil, dass durch weniger beachtete Positionen die benötigte Rechenleistung und Zeit, die benötigt wird sinkt. Dieser Ansatz ist zwar in der Lage in vielen Fällen, die notwendigen Kontexte erkennen, aber dies ist nicht in allen Fällen sicher. Dennoch ist dies durch die Vorteile ein Ansatz, der von einigen Unternehmen für ihre LLMs genutzt wird. [**?**, **?**]

Multi-Query Attention

Multi-Query Attention ist eine Weiterentwicklung der Multi-Head Attention. Dieser Mechanismus funktioniert ähnlich wie Multi-Head Attention, hierbei werden in dieser Variante jedoch die Key und Value Matrizen unter den verschiedenen Heads geteilt. Dadurch müssen diese großen Matrizen nicht öfter geladen werden und es wird an Speicher und an Zeit gespart. Dies sorgt vorallem für eine schnellere Inference. Hier gibt es auch nur eine kleine Verringerung der Qualität [**?**].

Grouped-Query Attention

Die Grouped-Query Attention ist eine weitere Variante der Multi-Head Attention. Diese ist eine Variante, die von der Multi-Query Attention abstammt und zwischen Multi-Head Attention und Multi-Query Attention liegt. Hier wird ebenfalls die Anzahl der Key-Value Heads verändert. In dieser Version gibt es davon mehr als eine und weniger als die Anzahl der Query Heads. Inference funktioniert hierbei schneller als bei der Multi-Head Attention und es geht weniger Qualität verloren als bei Multi-Query Attention [**?**].

2.3.2 Tokenizer

Tokenizer sind ein wichtiger Part der Machine Learning Toolchain im Zusammenhang mit dem Lernen von menschlicher Sprache. Wenn man im Bereich des Natural Language

Processing ein Machine Learning Modell trainieren möchte stellt sich die Frage, wie man Texte aufteilen kann, damit das Modell, welches trainiert wird diese verarbeiten kann.

Die Tokenizer teilen die Texte so auf, dass das zu trainierende Modell komplexe Sätze besser verarbeiten kann. Der Tokenizer macht das, indem er den Text in sogenannte Tokens aufteilt. Ein Token kann ein ganzes Wort, ein Teil eines Worts oder ein Satzzeichen sein.

Dies ist ein wichtiger Teil des Natural Language Processing, da die Aufteilung der Trainingsdaten einen großen Einfluss darauf hat, wie gut die Large Language Model funktioniert. Genau wie bei Compilern, die Programme in Tokens aufteilen, benutzen wir bei Large Language Model Tokenizer, damit wir die Texte aufteilen können und die dabei entstandenen Tokens über verschiedene grammatikalische Regeln zusammensetzen können [**?**].

Eines der größten Probleme hierbei ist, dass verschiedene Sprachen verschiedene grammatikalische Regeln haben und somit verschiedene Arten des Zerlegens oder Zusammensetzens brauchen. Ein weiteres Problem ist, dass das selbe Wort teils verschiedene Bedeutungen haben kann. Hierbei ist der Kontext wichtig und die Anzahl der Tokens, die das Modell erkennen kann. Man könnte hierbei alle möglichen Token aufnehmen, was aber dafür sorgen kann, dass das System overfitted. Dazu kommt, dass das die notwendigen Kosten zum Trainieren und für die Inference des Systems erhöhen würde [**?**, **?**].

2.3.3 Training

Das Training einer LLM hat zwei Schritte. Der erste Schritt ist das Pretraining, bei dem die LLM generell trainiert wird. Der zweite Schritt ist das Finetuning indem das vortrainierte Modell aus Schritt eins mit einem weiteren Datensatz trainiert wird und dadurch speziellere Kompetenzen erreicht, wie bessere Kenntnisse über eine bestimmte Domäne [**?**, **?**].

Pretraining

Das Pretraining einer LLM setzt die Grundlage für die LLM. Für das Pretraining einer LLM werden sehr viele Daten benötigt und diese sollten eine hohe Qualität haben. Bessere Qualität dieser Daten, hilft der LLM später bessere Ausgaben zu generieren. Daher ist es wichtig eine gute Datenpipeline für die Trainingsdaten vorzubereiten, welche die Datenqualität sicherstellen. [**?**, **?**]

Um ein etwas besseres Bild davon zu haben, wie viele Daten benötigt werden, wird Llama3 von Meta betrachtet. Llama3 wurde mit 15 Billionen Token trainiert [**?**].

Um einen Vergleich zu haben, wird der fineweb Datensatz, welcher auf Huggingface zur Verfügung steht, betrachtet. Dieser Datensatz hat ebenfalls 15T Token und hat eine Speichergröße von 47.6TB [**?**].

Für das Pretraining werden Daten aus den Bereichen benötigt, in denen die LLM später Dinge generieren soll. Ebenfalls die Sprache der Daten ist wichtig. Das Verhältnis der Menge der Daten zueinander beeinflusst ebenfalls die Performance der LLM.

Das Training ist der rechnerisch aufwändigste Teil der LLM Erstellung. Für das Training des Llama3 8B Parameter Models hat Meta 1.3 Millionen GPU Stunden gebraucht. Mit dem GPU Cluster, den Meta zur Vefügung hat, mit 24,576 H100 CPUs, hat es ca 2.2 Tage gedauert dies zu trainieren.[**?**, **?**, **?**]

In diesem Kontext gibt es auch verschiedene Unternehmen die Partnerschaften mit KI Unternehmen schließen und diesen Zugriff auf ihre Daten geben, damit sie besser in diesem Bereich werden. Ein Beispiel für so einen Zusammenschluss ist der Zusammenschluss von OpenAI und Stackoverflow, wobei OpenAI hofft, dadurch ein Model mit besseren Coding

Kompetenzen zu erhalten und Stackoverflow hofft, dass ihre Nutzer dadurch mehr mit Inhalt auf der Seite interagieren [**?**].

Finetuning

Beim Finetuning einer LLM wird auf dem vortrainierten Model aufgebaut und das Model wird mit einem kleineren spezifischeren Datensatz weiter trainiert. Dieser Schritt ist rechnerisch weniger aufwändig als das vortrainieren und ist daher für mehr Unternehmen und Instiutionen zugänglich. Hinzu kommt, dass dieser Schritt weniger zeitaufwändig ist [**?**].

Um einen Vergleich für den Aufwand zu haben wird "DiscoResearch/Llama3-German-8Bäuf Huggingface betrachtet. Das Model, welches 1.3 Millionen GPU Stunden für das Pretraining gekostet hat, wird jetzt gefinetuned. DiscoResearch hat hierfür Zugriff auf den Supercomputer 42 von hessian.Ai bekommen. Das Model wurde von 128 GPUs innerhalb von ca 60 Stunden über 65 Milliarden Tokens gefinetuned. Dies heißt, es hat ca 7680 GPU Stunden gekostet. Dieser Vergleich zeigt, dass dieser Schritt weniger aufwändig ist als das Pretraining [**?**].

Teilweise wird Finetuning verwendet um sogenannte Instruct Modelle zu erstellen. Diese Instruct Modelle sind Versionen des Standard Models, die darauf finetuned wurden, Instruktion zu befolgen. Bei einem Model, wie Llama3 von Meta, ist die nicht finetuned Variante des Models bereits in der Lage Instruktionen zu befolgen, wird aber durch ein kurzes Finetuning messbar besser in diesem Bereich [**?**].

Eine weitere Richtung in die Finetuning gehen kann ist das Trainieren eines Models auf einer neuen Sprache. Hierfür betrachten wir "DiscoResearch/Llama3-German-8Bäuf Hugginface. Eine auf deutsch fine-tuned Version von Llama3. Sie wurde mit Benchmarks veröffentlicht, die uns zeigen, dass die deutsche Leistung des Models sich durch das finetuning mittels 65 Milliarden Tokens verbessert hat. Die Englische Leistung ist etwas verschlechtert, aber vorallem beim hellaswag_de [**?**] Benchmark, sieht man, dass die Leistung sich messbar bessert. [**?**]

2.3.4 Inference & Generation

Sobald ein Model trainiert wurde und bereit ist benutzt zu werden, muss man es ausführen. Dieser Prozess nennt sich Inference. Um Inference ausführen zu können gibt es verschiedene Möglichkeiten. Es gibt die Möglichkeit Inference über online APIs wie beispielsweise die Huggingface Inference API [**?**] oder ähnliche Services auszuführen, oder man kann die Modelle lokal ausführen.

Inference

Wenn man eine der APIs nutzt um ein Model zu nutzen, dann kann man Anfragen an die API schicken und das Ausführen des Models wird von den Betreibern der API übernommen.

Die andere Option, das Model lokal auszuführen, kann über verschiedene Methoden passieren. Eine Option ist es einen Service wie Ollama (siehe **??**) oder LM Studio nutzen [**?**], worüber man die Modelle downloaden und ausführen kann. Mit Ollama kann man das Model über die CLI ausführen und mit ihr kommunizieren und über LM Studio kann man das mittels einer GUI machen. Beide Projekte basieren auf llama.cpp (siehe **??**).

Eine andere Option, das Model lokal auszuführen, ist es die Gewichte der Modelle zu downloaden und diese mittels Code auszuführen. Hierfür kann man die Modelle teils direkt

bei den Unternehmen downloaden wie bei Llama von Meta, aber eine andere Website auf der viele Modelle veröffentlicht werden, auf der man auch Vergleiche zwischen mancher dieser Modelle sehen kann ist hugginface.co (siehe **??**).

Andrej Karpathy [**?**] hat gezeigt, dass man mit einen 15 Millionen Parameter Model von Llama2 in C ohne externe Libraries innerhalb von unter 1000 Zeilen Inferenz betreiben kann [**?**]. Modelle die heute veröffentlicht werden haben aber weit über 15 Millionen Parameter und daher ist die Inferenz mit so einem simplen Programm nicht sehr effektiv. Das Llama2 Model mit 7 Milliarden Parametern generiert auf einem M1 Macbook Air einen Token pro 30 Sekunden. Die Modelle in einer Milliarden Größe, brauchen aufgrund ihrer Größe ein paar Optimierungen [**?**].

Eine Möglichkeit Modelle schneller zu machen ist es sie zu quantisieren, dieser Prozess wird in **??** genauer erklärt.

Statt den Code für die Inferenz selbst zu schreiben gibt es auch noch die Möglichkeit verschiedene Frameworks zu verwenden. Drei dieser Frameworks werden in **??** vorgestellt und erklärt.

Generation

Die Generation der LLMs kommt auf ihre Trainingsdaten an. Genau wie beim Training arbeiten die LLMs beim generieren ihrer Ausgaben in Tokens. Hierfür nutzen die Modelle ihre vorher durch Training antrainierten Parameter dazu den nächsten Token hervorzusagen.

Diese Vorhersage findet sequentiell statt und nutzt dabei die neu generierten Tokens bei der Vorhersage des nächsten Tokens. Hierbei werden alle bisherigen Tokens für die Generierung des nächsten Tokens berücksichtigt, bis das maximale Kontextfenster, welches in **??** beschrieben wird, erreicht wurde.[**?**].

Kontextfenster

Als Kontextfenster werden die Menge an Tokens bezeichnet, die ein ein Modell maximal zum Zeitpunkt der Generierung betrachten kann. Die Token außerhalb des Kontextfensters, werden bei der Generation nicht mehr berücksichtigt. Das Kontextfenster ist die maximale Eingabesequenzlänge des Transformers **??**.

2.3.5 Limitationen & bekannte Probleme

LLMs haben trotz ihrer ständigen Verbesserungen einige Probleme, die schon seit einige LLM Generationen bekannt sind.

Ein Problem, welches LLMs haben ist, dass die Informationen, die LLMs haben, teils veraltet ist. Dies ist darauf zurückzuführen, dass die LLMs zwar mit sehr vielen Datensätzen trainiert werden, welche aber zeitlich, bis zum Zeitpunkt des Trainings, begrenzt sind. Der Trainingsprozess ist zeit- und ressourcenintensiv, während dem Training und dem Finetuning des Modells entsteht eine Lücke zwischen dem Wissensstand des Modells und der realen Welt. Um dieses Problem zu lösen gibt es Ansätze, wie RAG, welches in **??** näher erklärt wird.

Ein weiteres Problem, ist die feste Größe des Kontextfensters der LLMs. Dies führt je nach Aufgabe dazu, dass Kontexte nicht beachtet werden oder Informationen, die zu weit zurück liegen, nicht beachten werden können **??**.

Obwohl LLMs mit sehr vielen Datensätzen trainiert werden, ist ein Problem, dass LLMs halluzinieren können. Das heißt, das Modell generiert plausibel klingende Informationen, die aber faktisch nicht korrekt sind, entweder im Zusammenhang mit der Eingabe des Nutzers oder der zum Training genutzten Daten [**?**].

Das letzte Problem, welches in dieser Arbeit besprochen wird ist der Reversal Curse. Dies ist ein Problem der LLMs im Zusammenhang mit Generalisierung von Sprache. Der Reversal Curse tritt auf, wenn ein Modell mit einem Satz der Form "A ist B" trainiert wird. Manchmal ist die LLM nicht in der Lage durch diesen Satz die umgekehrte Schlussfolgerung zu lernen. Hier ist ein bekannter Fall in GPT-4 in dem GPT-4 die Frage "Wer ist die Mutter von Tom Cruise?" in 79% der Fälle richtig mit "Mary Lee Pfeiffer" beantwortet und die Frage "Wer ist der Sohn von Mary Lee Pfeiffer?" nur in 33% der Fälle richtig beantwortet [**?**].

2.3.6 Prompting

Ein Prompt im Bereich der LLMs ist die Texteingabe des Nutzers, welche die Ausgabe des LLMs bestimmt. Die Art und Weise der Eingabe kann von LLM zu LLM unterschiedlich sein. Bei Modellen, welche Bilder generieren, wie Midjourney oder DALLE-3 sind die Prompts eher beschreibend und bei Prompts für Chatbots, wie ChatGPT sind die Prompts eher Fragestellungen oder Anfragen [**?**].

Die Prompts können starken Einfluss auf die Qualität der Ausgaben des Systems haben. Es gibt verschiedene Methoden, die mit der Zeit gefunden wurden um die Ausgaben des Systems zu verbessern. Aufgrund dessen hat sich im Bereich der LLMs eine neue Disziplin namens Prompt Engineering gebildet. Innerhalb dieser geht es darum, Prompts so zu optimieren, dass das LLM System genau das macht was der Nutzer will [**?**].

Egal wie gut eine Prompt ist, manchmal kann eine LLM Applikation die gewollte Ausgabe nicht ausgeben. Dies kann verschiedene Gründe haben, wie zum Beispiel fehlende Informationen, die Informationen können fehlen, weil sie nicht mit in den Trainingsdatensatz der LLM aufgenommen wurden, obwohl es sie schon gab, aber es könnte auch sein, dass die Informationen erst veröffentlicht wurden, nachdem die LLM zu Ende trainiert wurde. Aufgrund dessen ist es wichtig, die Limitationen der LLM, die verwendet wird, zu kennen [**?**].

Es gibt zwar keinen eindeutigen Weg gute Prompts zu generieren, aber es gibt ein paar Best Practices, welche generell anerkannt sind. Im Prompt nur die Aufgaben mitteilen nennt sich Zero-Shot Prompting. Das heißt es werden keine Beispiele mitgegeben und manche LLMs sind in der Lage Zero-Shot prompts in manchen Aufgabenbereichen zu lösen. Der LLM im Prompt ein Beispiel für die gewollte Ausgabe mitzugeben nennt sich One-Shot Prompting und mehrere mitegeben heißt few-shot prompting. Die Qualität der gewollten Ausgaben lässt sich teils mithilfe von One-Shot oder Few-Shot Prompts statt Zero-Shot Prompts verbessern. Jedoch hat der Input Kontext eine begrenzte Größe und umso größer der Input ist, umso größer ist der computational(was heißt das auf deutsch?) Aufwand, daher ist das eine begrenzte Lösung, die nicht alle Probleme beseitigen kann [**?**]. Ein paar der davon sind:

- Anweisungen an den Anfang oder das Ende des Prompts stellen.
- Spezifische Anweisungen und genaue Beschreibungen geben. Auch spezifisch und genau bei den gewollten Ausgaben sein.

- Anweisungen die der LLM mitteilen was sie tun soll, statt den Dingen die sie nicht tun soll, sind zu bevorzugen.

Es gibt auch ein paar fortgeschrittene Methoden die verwendet werden können [**?**]. Hierzu gehören:

- Der LLM "kraftvoll"mitteilen, was man von ihr möchte.
- Gedankengang Prompting, bei der man die LLM, die einzelnen Schritte die sie gemacht hat und die Gründe dafür erklären lässt.

Es gibt eine spezielle Art der Prompts namens System Prompt. Dieses System Prompt dient dazu das Verhalten der LLM in einem Rahmen zu steuern. Durch dieses System Prompt kann der LLM mitgeteilt werden, welche Rolle sie einnimt und unter anderem auch ihren Stil beeinflussen.

2.4 Vektor Datenbanken

Vektor Datenkbanken sind Datenbanken, die hochdimensionale Vektoren speichern können. Diese Vektoren sind mathematische Repräsentationen von Attributen oder Merkmalen. Je nachdem wie granular die Daten, die abgespeichert werden sollen sind, kann es sein, dass die Daten tausende Dimensionen haben. Die Rohdaten werden mittels Einbettungsfunktion oder Transformationen in ihre Vektordarstellung umgewandelt und dann abgespeichert.

Diese Art der Datenbanken hat für den AI und Machine Learning Bereich ein paar Vorteile im Vergleich zu traditionellen Datenbanken.

Ähnlichkeitssuche: Traditionell Datenbanken brauchen klar definierte Kriterien für die Suche. Hier ist es nicht möglich eine Ähnlichkeitssuche, wie bei Vektordatenbanken durchzuführen. Bei Vektordatenbanken kann man mittels Ähnlichkeitssuche-Verfahren schnell ähnliche Datensätze finden und dann ausgeben.

Support für unstrukturierte und komplexe Daten: Traditionelle Datenbanken brauchen strukturierte Daten. Die Daten die man für Machine Learning benutzt sind oft unstrukturierte Daten, die verschieden Formate haben oder komplett verschiedene Datentypen sind, dies schließt auch Audio und Videodateien mit ein. Vektordatenbanken können diese Dateien durch die Vektordarstellung abspeichern und sind damit besser für diesen Fall geeignet.

Skalierbarkeit und Performance: Dadurch, dass Vektordatenbanken oft mit Big Data verwendet werden, gibt es einige Ansätze zum Verbessern der Performance und Skalierbarkeit. Ein paar der Methoden die verwendet werden sind Partitioning, Sharding und Caching. Wie genau diese Methoden funktioniert sind irrelevant für diese Arbeit und werden daher nicht weiter angesprochen.

2.5 Embedding Modelle

Embedding Modelle dienen dafür Daten in eine Vektorrepräsentation umzuwandeln. Diese Modelle sind in der Lage verschiedene Datenformate umzuwandeln. Unter anderem Text, Video und auch Bilder.

Diese Vektoren sind hochdiemnsional und sollen dadurch auch Nuancen und die Bedeutung der Wörter mit in die Vektorrepräsentation aufnehmen. Dadurch, dass der Text in die jeweilige Vektorrepräsentation umgewandelt wird, können nach der Umwandlung

verschiedene mathematische Verfahren auf diesen Vektor angewendet werden. Dazu zählen Verfahren wie Semantische Suche und Semantische Nähe. Dies sorgt ebenfalls dafür, dass das Arbeiten mit den Daten schneller ist.

Embedding Modelle werden in LLM Applikationen dafür benutzt, Daten in ihre Vektorrepräsentation umzuwandeln und diese Vektoren zusammen mit den dazugehörigen Daten in einer Vektordatenbank zu speichern. [**?**, **?**, **?**]

Semantische Suche

Bei der semantischen Suche handelt es sich um ein Verfahren, welches dafür genutzt wird Daten semantisch zu vergleichen. Die Daten werden hierfür in ihre Vektordarstellung umgewandelt und dann werden die Vektoren miteinander verglichen und die top k Vektoren mit der höchsten Semantischen Nähe werden zurückgegeben.

Hierfür gibt es verschiedene Verfahren, drei Verfahren, die oft verwendet werden sind:

- Euklidischer Abstand
$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{2.1}$$
- Kosinus-Ähnlichkeit
$$\cos(\theta) = \frac{\sum_{i=1}^{n} p_i q_i}{\sqrt{\sum_{i=1}^{n} p_i^2} \sqrt{\sum_{i=1}^{n} q_i^2}} \tag{2.2}$$
- Manhattan-Metrik
$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i| \tag{2.3}$$

Mittels einer dieser oder eines anderen Verfahrens wird dann die Semantische Nähe berechnet und es werden die passenden Dateien, falls es diese gibt, zurückgegeben. Dies hilft es den LLM Applikationen dabei, passende Daten schnell abzurufen [**?**, **?**, **?**].

2.6 RAG

Eines der größten Probleme der LLMs heutzutage ist, dass die Informationen, die sie zum beantworten für Fragen benutzen, teilweise veraltet sind oder sehr allgemein. Ein weiteres großes Problem ist die im Abschnitt **??** genannte Halluzination, die passieren kann, wenn eine LLM eine Frage zu der es eigentlich nichts weiß, beantwortet. Eine Technologie, die versucht, dieses Problem zu beseitigen, ist die sogenannte Retrieval Augmented Generation. Die Retrieval Augmented Generation soll das parametrische Gedächtnis der LLM mittels eines nicht parametrischen Gedächtnisses erweitern.[**?**]

Retrieval Augmented Generation löst ein paar Probleme vor denen LLMs heutzutage stehen, ist aber keine perfekte Lösung und muss sich auch seinen eigenen Problemen stellen. In diesem Zusammenhang gibt kann man heute RAG in verschiedene Stufen einordnen.[**?**]

2.6.1 Naive RAG

Naive RAG ist die traditionelle Herangehensweise and das Problem der RAG. Sie besteht aus drei Stufen. Indexing, Abfrage und Generierung.

Beim Indizieren werden die rohen Eingabedaten gereinigt und extrahiert. Dies ist unter anderem wichtig, da die Daten verschiedene Formate haben können, wie PDF Dateien, Markdown oder HTML. Diese Texte werden nach der Reinigung in Plain Text umgewandelt und dieser Plain Text wird dann in kleinere Teile unterteilt. Diese kleineren Teile werden dann über ein Embedding modell in eine Vektordarstellung umgewandelt und dann in einer Vektordatenbank abgespeichert.[**?**]

Bei der Abfrage erhält das RAG System eine Anfrage und benutzt das selbe Embedding Modell, wie in der Indizierungsphase, um die Abfrage in eine Vektordarstellung umzuwandeln. Im Anschluss wird durch einen Semantischen Vergleich nach den Top-k Ergebnissen gesucht und diese werden dann ausgegeben.[**?**]

Bei der Generation bekommt die LLM dann die Abfrage und die ausgegeben Ergebnisse in einem Prompt zurück und generiert daraus dann eine Antwort. Was genau hier beachtet wird kommt auf die LLM, das Kontext Fenster und den benutzten Prompt an[**?**].

Wie in **??** bereits erwähnt gibt es hierbei ein paar Probleme. Das Hauptproblem, welches hier besteht ist die der Abfrage. Bei der Abfrage gibt es oft Probleme, welche mit der Genauigkeit zu tun haben. Dementsprechend kann es passieren, dass das RAG System falsche oder unpassende Informationen zurück gibt. Aufgrund der Probleme bei der Abfrage entstehen dann ebenfalls Probleme bei der Generation der Antwort. Was dazu führt, dass die Qualität der Antworten sinkt. Dadurch sinkt dann ebenfalls die Verlässlichkeit auf das System[**?**].

2.6.2 Advanced RAG

Advanced RAG wurde entwickelt um den Problemen der naiven RAG entgegenzuwirken bzw. sie zu beseitigen. Dementsprechend werden hier einige Dinge geändert. Vorallem in der Stufe der Abfrage wurde hier einiges geändert. Um die Qualität zu erhöhen wurden der Abfrage pre- und post Strategien hinzugefügt. Dazu wurde der Indexing Prozess angepasst. Beim Indexing wird nun Metadata mit eingebunden und der Text wird genauer unterteilt.

Pre-Retrieval

Beim pre-retrieval wird vorallem darauf geachtet die Anfrage so klar wie möglich zu machen. Hierfür werden Metadaten angehängt, Anfragen umgewandelt, Anfragen erweitert und noch einige weitere Dinge abgeändert. Das Ziel hier ist es die Anfrage zu ändern, sodass sie so klar wie möglich ist und sie für Retrival zu optimieren.

Post-Retrieval

Nachdem der Kontext nach dem Retrieval Prozess abgefragt wurde, wird versucht die abgerufenen Daten möglichst optimal mit der Anfrage zu verbinden. Hierfür werden vorallem das komprimieren und das neu Anordnen des Kontexts verwendet. Die wichtigsten Dokumente werden an eines der Enden der Anfrage geschoben und es wird versucht die wichtigsten Informationen heraus zu suchen und nur diese zurück zu geben. Dadurch sollte die LLM genug Informationen erlangen, aber nicht zu viele und die Antworten sollten besser werden[**?**, **?**].

2.7 llama.cpp

Ein Problem, welches im Abschnitt **??** bereits angesprochen wurde, ist, dass es schwierig sein kann, für Personen oder Unternehmen, die nicht über die nötige Hardware verfügen, LLM Inferenz zu betreiben. Desto größer die LLM, die benutzt werden soll, wird, desto aufwändiger werden die Anforderungen an die Hardware. Ein großes Open Source Projekt, welches sich mit diesem Problem auseinander setzt ist llama.cpp, gestartet von Georgi Gerganov auf Github.

Dieses Projekt hat zu Beginn viele Nutzer angezogen, da man hier, ohne spezielle Hardware, Inferenz betreiben konnte. Die anderen Bibliotheken, die hierfür zu Verfügung standen, wurden oft in Verbindung mit Closed-Source Bibliotheken, wie zum Beispiel Nvidias Cuda entwickelt und waren dadurch nur über bestimmte Hardware zu benutzen.

Zu Beginn wurde das Projekt hauptsächlich auf Inferenz auf der CPU ausgelegt. Dies gab den Nutzern zwar die Möglichkeit auf ihren Maschinen ohne GPU Inferenz zu betreiben, aber durch fehlende GPU Einbindung hatte dies Begrenzungen in der Geschwindigkeit und der Größe der Modelle, die benutzt werden konnten.

Mit der Zeit wurde dem Projekt auch die Möglichkeit hinzugefügt, GPUs für die Inferenz zu benutzen, wodurch die Nutzer in der Lage sind schnellere Inferenz mit größeren Modellen auszuführen.

Damit die verschiedenen Modelle über llama.cpp nutzbar sind, gab es hier zu Beginn das GGML format. Dies wurde ebenfalls von Gerganov im Zusammenhang mit seinen Projekten entwickelt.Ein Vorteil hierbei war, dass das GGML Format das speichern von quantisierten Modellen erlaubt. Quantisierte Modelle haben hierbei den Vorteil, dass die Präzision der einzelnen Parameter von bspw. 32 Bit Gleitkommazahlen auf eine niedrigere Genauigkeit, wie bspw. 8 Bit Gleitkommazahlen konvertiert wurde. Der Vorteil hier war, dass die Inferenz der einzelnen Modelle hier weniger intensive Rechenarbeiten erfordert haben und auch durch die kleinere Größe auch weniger Speicher benötigt. Dadurch war es einfacher möglich, die Inferenz auf der CPU durchzuführen.

Jedoch hatte diese Format einige Probleme. Zum einen hat jede Architektur ihr eigenes Skript benötigt, damit sie in das GGML Format konvertiert werden kann, dazu kam, dass in der Datei nicht erkennbar war, für welche Architektur das Modell war. Heutzutage benutzt llama.cpp ein neues Format, namens GGUF, welche einige der Probleme mit GGML beseitigen soll. Ein großer Vorteil von diesem Format ist, dass das Modell nun alle Informationen die zum laden des Modells notwendig sind beinhaltet und es müssen keine zusätzlichen Informationen mehr vom Nutzer gegeben werden. Unter anderem beinhaltet die Datei nun die Architektur des Modells

[**?**, **?**].

2.8 Ollama

Ollama ist ein Open Source Projekt, welches seinen Nutzern dabei helfen soll, auf der Plattform LLMs schneller auf ihren lokalen Maschinen zu starten und mit diesen zu interagieren. Das Backend das Ollama unterstützt ist llama.cpp und es soll das Downloaden, Installieren und das Interagieren mit den LLMs einfacher machen.

Ollama hat seine eigene Modellbibliothek über die Nutzer LLM Modelle downloaden können und diese dann direkt nutzen. Wenn ein Modell nicht in der Bibliothek zu finden ist, kann man Modelle mittels Modelfile selbst importieren. Dies kann man mit Modellen im

GGUF Format, welches in **??** genauer erklärt wurde, gemacht werden und mit sogenannten safetensor Dateien[**?**]. Das Safetensor Format wurde von Huggingface eingeführt um die Unsicherheiten des vorher genutzten Pickle Formats zu beseitigen. Eines der größten Probleme dieses Formats war es, dass beim deserialisieren eines Pickle Objekts boshafter Code ausgeführt werden konnte, da Pickle den unterschied zwischen boshaften und nicht boshaften Code nicht erkennen kann.[**?**] Mit dem neuen Safetensor Format konnte vorallem dieses Problem beseitigt werden.

Ollama kann Safetensor Dateien für die drei Architekturen LlamaForCausalLM, MistralForCausalLM und GemmaForCausalLM ausführen. Wenn man ein Modell hat, welches nicht das GGUF Format oder das Safetensor Format für eine der drei genannten Architekturen ist dann muss man dieses Modell in GGUF umwandeln. Dafür kann man in das llama.cpp Github Projekt und das convert script benutzen.

Im Anschluss kann man dann das Modell mithilfe einer Modelfile importieren. Innerhalb dieser Modelfile kann man ebenfalls noch weitere Dinge vorher festlegen, wie zum Beispiel das System Prompt für das Model[**?**].

2.9 Ragas

2.10 AI Frameworks

Dadurch, dass künstliche Intelligenz nun ein größeres Thema in den Augen der Öffentlichkeit ist und es durch Bibliotheken wie llama.cpp einfacher geworden ist, für Personen ohne besondere Hardware LLMs zu nutzen, gibt es immer mehr Entwickler, die Applikationen, die LLMs nutzen, entwickeln wollen.

Da es immer mehr Applikationen dafür gibt, gibt es auch einige KI Frameworks, die das Arbeiten mit diesen Modellen vereinfachen sollen. Ein großer Fokus bei den meisten der KI Frameworks liegt darauf den Nutzern das Arbeiten mit eigenen Daten einfacher zu machen indem beispielweise eine RAG Pipeline genutzt wird. Hier werden einige der größten Frameworks vorgestellt.

2.10.1 Haystack

Haystack ist ein Open Source Framework, welches LLM Applikationen erstellen kann. Der Fokus hier liegt vorallem auf RAG Applikationen und Such Systemen, welche mit großen Mengen an Daten arbeiten können.

Bei Open Source Frameworks ist die Community einer der wichtigsten Faktoren. Umso mehr Teilnehmer an einem Open Source Projekt es gibt, umso mehr Unterstützung gibt es und umso mehr Fehler fallen den Leuten auf, die dann beseitigt werden können. Haystack hat 14.3k Sterne auf Github, zum Zeitpunkt dieser Arbeit. Das macht es zum kleinsten Projekt auf dieser Liste.

Haystack kann direkt Modelle von Huggingface, OpenAI und anderen Quellen beziehen und diese ausführen. Dazu kann man Modelle nutzen die auf Hosting Plattformen, wie Azure oder Sagemaker gehostet werden.

Die zwei Konzepte die hierbei am wichtigsten sind sind Komponenten und Pipelines. Komponenten sind Blöcke, welche Aufgaben durchführen können. Aufgaben, die eine Komponente ausführen kann, sind zum Beispiel Text Generation und das Erzeugen von Einbettungen.

Eine Pipeline besteht aus verschiedenen Komponenten. Man setzt diese Komponenten zusammen und erhält dadurch eine Pipeline. Dadurch kann man den Datenfluss innerhalb seiner Applikation genau beeinflussen.

Die Programmiersprache mit der Haystack arbeiten kann ist Python[**?**, **?**].

2.10.2 Langchain

Langchain ist ein weiteres Open Source Framework, welches für das Erstellen von LLM Applikationen erstellt wurde. Im Gegensatz zu Haystack ist Langchain nicht nur auf das Erstellen von RAG Applikationen und Such Systemen fokussiert. Langhchain versucht möglichst vielseitig einsetzbar zu sein. Öne Framework. Infinite Use Cases"werben sie auf ihrer Homepage.

Langchain ist, in Hinsicht auf die Github Sterne das größte Framwork, welches hier vorgestellt wird. Zur Zeitpunkt des Schreibens dieser Arbeit, hat das Projekt 87.6k Sterne auf Github.

Für die Entwicklung liegt der Fokus bei Langchain auf Chains. Welche ähnlich wie Pipelines den Datenfluss bestimmen. Man erstellt hier die einzelnen Teile die man braucht, wie einen Retriever der mit einer Vektor Datenbank verbunden ist und einer LLM. Diese verbindet man dann noch mit der Eingabe des Nutzers und hat dann eine fertige Chain, die man dann einsetzen kann, wie man möchte. Eine Möglichkeit wäre es diese per RestAPI verfügbar zu machen.

Dazu hat gibt es das Paket Langserve, welches darauf ausgelegt ist, einfach Chains als RestAPIs zu veröffentlichen. Dazu kommt ein Paket namens Langsmith, welches zum Monitoring und zum Evaluieren von LLM Applikation dient. Langsmith ist nicht Open Source und kostet für Teams oder für Hobby Entwickler ab einer gewissen Menge an Nutzungen ein monatliches Abbonement.

Langchain ist ein Framwork, welches für Javascript und für Python zur Verfügung steht. [**?**, **?**]

2.10.3 LlamaIndex

LlamaIndex ist ein weiteres Open Source Framework, das entwickelt wurde, um die Erstellung von LLM Applikationen zu erleichtern.

Zum Zeitpunkt des Schreibens dieser Arbeit hat das Projekt 32.8k Sterne auf GitHub und ist damit das zweitgrößte Projekt, welches hier vorgestellt wird, aber es ist trotzdem weniger als halb so groß, wie Langchain.

Bei LlamaIndex liegt der Fokus auf dem Arbeiten mit Daten. Deshalb ist einer der Fokus Punkte des Frameworks der Index. RAG Use Cases in LlamaIndex benutzen in ihrem Kern den Index als Kernkonzept. Indexe werden aus Dokumenten erstellt und sind im Grunde Knoten-Objekte, welche Teile von den Originaldokumenten respräsentieren.

LlamaIndex hat eine Plattform namens LlamaCloud, welche einen durch generative künstliche Intelligenz angetriebenen Parser hat. Dieses Projekt heißt LlamaParse und lässt sich mit LlamaIndex integrieren. LlamaParse kann man auch ohne die LlamaCloud nutzen. Wenn man die LlamaCloud nutzt hat man noch die Option seine RAG Daten dort über eine API zu speichern. Über die API kann man sich direkt mit verschiedenen Datenquellen verbinden.

LlamaIndex ist ein Framework, welches, wie Langchain, für Javascript und für Python zur Verfügung steht[**?**, **?**, **?**, **?**].

2.11 Huggingface / MTEB

Huggingface ist eine Plattform, auf der verschiedene Daten gehostet werden, Diskussionen stattfinden und KI Apps zu finden sind.

Auf dieser Plattform werden Daten in Git basierten Repositories gelagert. Zu den Daten die hier zu finden sind, gehören unter anderem Open Source LLM Modelle, Open Source Embedding Modelle und Datensätze zum trainieren von solchen Modellen. Es gibt die Möglichkeit KI-basierte Demo Apps in sogenannten Spaces auszuprobieren oder zu hosten. Hinzu kommt es die Möglichkeit Beiträge zu veröffentlichen, ähnlich wie auf Social Media Plattformen, wie X oder Threads.

Neben den Demo Apps die in Spaces zu finden sind, werden auch Ranglisten veröffentlicht. Eine der größten Ranglisten ist die MTEB Rangliste. MTEB steht für Massive Text Embedding Benchmark. Diese Rangliste wurde für Einbettungsmodelle erstellt und wird regelmäßig geupdated. Die Modelle werden hier in 8 verschiedenen Aufgabenbereichen verglichen.

Ein paar der Aufgabenbereiche sind Clustering, Semantische Nähe und Zusammenfassungen. Die Datensätze wurden so ausgewählt, dass es eine verschiedene Länge an Texten gibt, damit die verglichen werden kann, wie die Modelle mit kurzen und auch mit langen Texten umgehen können. Aktuell ist das beste Modell im Durchschnitt in dieser Rangliste das Modell "SFR-Embedding-2_R" von Salesforce Research [**?**, **?**].

2.12 Google Chat

Google Chat ist eine von Google entwickelte Chatplattform. Diese Plattform hat 2017 Google Hangout zusammen mit Google Meet ersetzt. Zu Beginn war das Projekt nur im Zusammenhang mit Google Workspace erhältlich und somit war es eine Chatplattform die hauptsächlich für Teams und Unternehmen benutzbar war. 2021 wurde die Plattform im Early Access auch für normale Benutzer erreichbar gemacht und ist seit 2022 für jeden benutzbar.

Diese Plattform ist die Plattform, die die Techdivision hauptsächlich intern zur Kommunikation benutzt. Es gibt auf der einen Seite 1 zu 1 Chat in der eine Person mit nur einer anderer Person kommuniziert und auf der anderen Seite gibt es Innerhalb dieses Services die Möglichkeit sogenannte Spaces zu erstellen, welche Gruppenchats ähneln. Ein Teil dieser Gruppenchats sind die in **??** genannten Supportchats in die der Chatbot final eingebaut werden soll.

3 Anforderungsanalyse

Techdivision

Die Techdivision GmbH ist ein IT-Dienstleister aus Kolbermoor. Sie bietet Leistungen in vier Kernbereichen.

1. Consulting

2. Software & Solutions
3. Daten & Prozesse
4. Marketing & Kreation

Über diese vier Bereiche möchte die Firma den Kunden bei der digitalen Transformation helfen. Innerhalb jedem dieser Bereiche gibt es verschiedene Teams die sich um spezifische Aufgaben des Teilbereichs kümmern.

Die Mitarbeiter der Techdivision kommunizieren, wenn nicht in Person, über den Google Chat. Die einzelnen Fachbereiche haben eigene Spaces innerhalb des Google Chats. Einer dieser Chats ist der IT-Support Chat, in dem Mitarbeiter technische Probleme melden oder bei technischen Problemen Fragen stellen können und für diese Unterstützung vom IT-Support erhalten.

3.1 Ausgangslage

Die aktuelle Ausgangslage des IT Supports innerhalb der Techdivision ist, dass der IT-Support hauptsächlich über den internen Google Chat betrieben wird oder über IT-Helpdesk Tickets. Die Tickets werden verwendet, wenn es Probleme sind, welche keine akute Hilfe benötigen. Probleme, welche akute Hilfe benötigen, sind Probleme, die zeitkritisch und Blocker sind. Blocker sind Probleme, durch die nicht weiter gearbeitet werden kann.

Wenn es akute Probleme sind, dann meldet man sich beim IT-Support Chat. Dies ist eine offene Chatgruppe für Mitarbeiter der Techdivsion und innerhalb diese können Fragen gestellt werden und sie werden von einem der Mitarbeiter des IT-Support Teams beantwortet. Falls es fragen sind, welche nicht über den Chat gelöst werden können, dann wird ein Google Meet Meeting aufgesetzt oder, falls der jeweilige Mitarbeiter sich im Office befindet, dann kann das auch vor Ort stattfinden.

Informationen und How To's der Techdivsion sind in den verschiedenen Datenquellen der Techdivsion zu finden.

Hierfür gibt es ein Confluence System. Hier werden die Daten nach Bereichen geordnet, wobei verschiedene Teams verschiedene Bereiche haben. Der Bereich, der für diese Arbeit am relevantesten ist ist der DevOps Bereich. In diesem werden verschiedene Technologien erklärt und mittels Anleitungen wird erklärt, wie man diese einrichtet oder typische Probleme löst. Dazu gibt es ein internes Stackoverflow, in dem die Mitarbeiter Fragen stellen können und andere Mitarbeiter können diese beantworten oder kommentieren. Dazu gibt es verschiedene Dateien in Google Drive, die abgespeichert sind.

3.2 Anforderungen

Nun wird gefordert, dass dem IT-Support Gruppenchat ein Chatbot hinzugefügt wird, der den Mitarbeitern des Support Teams helfen soll, indem er einfache Fragen, die bspw. mithilfe von Dokumentationen innerhalb des internen Confluences beantwortbar wären, beantworten soll.

Im Folgenden werden die Anforderungen, die der Chat erfüllen sollte vorgestellt.

3.2.1 A1 Selbst Hosting

Das größte Thema, welches beim Betreiben des Chatbots beachtet werden soll, ist der Datenschutz. Datenschutz ist ein großes Thema bei Unternehmen und vor allem, bei Unternehmen die mit Kundendaten arbeiten müssen oder auch bei internen Betriebsgehemnissen, die nicht an die Öffentlichkeit weiter gegeben werden sollen.

Die Techdivision ist ein IT-Dienstleister und somit ist dies auch ein Bereich der bei der Konzeption und Entwicklung dieses Chatbots beachtet werden muss. Aufgrund dieser Sorgen wird gefordert, dass der Chatbot von der Techdivision gehostet wird, was bedeutet es dürfen keine APIs von externen LLM-Systemen hierfür verwendet werden, die von anderen Unternehmen gehostet werden, wie beispielsweise die ChatGPT API.

3.2.2 A2 Integration in Google Chat

Das Hauptkommunikationsmittel, welches innerhalb der Techdivsion genutzt wird ist Google Chat. Teilweise findet Kommunikation auch über Email statt, jedoch ist das nur ein kleiner Teil.

Dazu kommt, dass der IT-Support innerhalb der Techdivsion hautpsächlich im Google Chat stattfindet

3.2.3 A3 Gruppenchat-Funktionalität

Auch, wenn die LLMs heutzutage weit entwickelt sind, können diese die falschen Informationen weiter geben, oder komplett neue halluzinieren und diese dann dem Supportsuchenden weiter geben. Es könnte auch sein, dass der Bot dem Mitarbeiter überhaupt nicht helfen kann. Damit in diesen Fällen ein Supportmitarbeiter eingreifen kann, sollte der Bot in einer Chatgruppe laufen, von der ein Mitarbeiter des Support Teams ebenfalls Teil ist.

3.2.4 A4 Nutzung von Unternehmensdaten

Da die LLMs, die online auffindbar sind, keinen Zugriff auf spezifische Informationen der Techdivion haben, soll der Bot auf Unternehmensdaten zugreifen können. Dadurch kann der Bot direkt spezifischere Hilfe leisten und nicht nur allgemeine Hinweise geben, die im Internet zu finden sind, sondern hat auch Zugriff auf Dinge wie Anleitungen aus dem internen Confluence.

3.2.5 A5 Erweiterbarkeit der Wissensdatenbank

Da sich die Unternehmensdaten mit der Zeit ändern und auch neue Datenquellen hinzukommen können, sollte der Bot in der Lage sein mit der Zeit auf neues Wissen zuzugreifen.

Das bedeutet, die Wissensdatenbank des Chatbots soll kontinuierlich erweitert werden und bei Updates in den Datensystemen der Techdivsion regelmäßig aktualisiert werden.

3.2.6 A6 Deutschsprachige Interaktion

Der Großteil der Techdivion ist deutschsprachig. Die meisten Teile von Datenquellen, wie beispielsweise Confluence sind ebenfalls auf Deutsch. Es gibt aber auch englischsprachige Mitarbeiter, daher wäre ein Bilingualer Bot der beste Fall, da aber die meisten Deutsch sind, wäre Deutsch die bessere Option, falls es nur eine Sprache sein kann.

3.2.7 A7 Logische & schlüssige Argumente

Die Argumente des Chatbots sollen logisch und schlüssig sein. Diese Anforderung hängt mit **??** zusammen. Der Chatbot soll die Fragen logisch und schlüssig ohne Halluzination mit Hilfe der Unternehmensdaten beantworten.

3.2.8 A8 Speicherung von Anfragedaten

Dies ist eine optionale Anfrage des IT-Supports. Der Bot soll Anfragedaten speichern können, damit eingesehen werden kann, ob zu manchen Themen viele Fragen gestellt werden oder manche Personen besonders viele Fragen stellen. So könnte, falls notwendig zu einem bestimmten Thema ein Workshop angeboten werden oder es könnte ein Kurs mit einer Person geben, in der diese Person alles, was sie will erklärt kriegt.

4 Konzeption

Bevor mit der Entwicklung begonnen werden kann, muss das Konzept erstellt werden. Zur Konzeption gehört in dieser Arbeit, die Auswahl der technischen Werkzeuge, die Programmiersprache, die Umgebung, die Modelle die benutzt werden sollen und die Architektur des Systems.

4.1 Auswahl der Modelle

Die erste Frage, die beantwortet werden muss, ist, welche Modelle werden benutzt. Da innerhalb dieser Arbeit LLMs miteinander verglichen werden, werden fünf Modelle benötigt. Vier LLMs für den Chatbot selbst, der die Nachrichten generieren soll, die im Anschluss miteinander verglichen werden und ein Embedding Modell, welches für die Einbettungen der Vektordatenbank und der Anfragen zuständig sein wird.

4.1.1 Auswahl der Chat LLMs

Als erstes werden die Chat LLMs ausgwählt. Die Anforderung die hier beachtet werden muss, ist A1. Da der Bot lokal laufen soll, werden keine Modelle benutzt, die nur über eine externe API erreichbar sind, wie OpenAIs ChatGPT. Also ist die einzige Option ein Modell zu downloaden und mit diesem lokal zu interagieren.

In dieser Arbeit werden 4 verschiedene Modelle miteinander verglichen. Ein Hub auf dem viele der Modelle veröffentlicht werden ist hugginface.co (siehe hierzu **??**).

Innerhalb dieser Arbeit werden nur Modelle unter 14B Parametern miteinander verglichen, da die Generationszeit kleinerer Modelle schneller ist und die meisten Informationen mit denen das Modell arbeiten soll innerhalb der Vektordatenbank gespeichert sein werden und über RAG dem Modell übergeben werden. Für diese Arbeit wurden die drei Modelle, die auf Ollama die meisten Downloads haben

1. Mistral 7B von Mistral AI

2. Llama3 8B von Meta AI
3. Gemma 2 9B von Google

ausgewählt. Dazu kommt das zur Zeit der Auswahl neu veröffentlichte Phi3 Medium mit 14B Parametern von Microsoft als letztes Modell dazu. Diese Modelle wurden mit vielen Daten trainiert und dazu auch mit mehrsprachigen. Daher sind diese Modelle ohne weiteres Finetuning bereits in der Lage auf deutsch mit dem Nutzer zu interagieren und dadurch wird Anforderung A6 erfüllt.

4.1.2 Auswahl der Embedding Modelle

Nach der Auswahl des Chatmodelle geht es um die Auswahl des nächsten Modells. Es wäre hier möglich das extra Einbettungsmodell weg zu lassen und es mit den Chatmodellen einbetten zu lassen. Damit der Vergleich zwischen den verschiedenen Chatmodellen nicht durch die Einbettungen beeinflusst wird, wird für alle Chatmodelle das selbe Einbettungsmodell verwendet.

Ebenfalls, wie bei den Chat Modellen werden hier Open Source Modelle benötigt, welche auf einem eigenen Server gehosted werden können.

Hierfür wird das Massive Text Embedding Leaderboard (MTEB) auf Huggingface betrachtet. Das MTEB Leaderboard sortiert Embedding Modelle nach ihrem durchschnittlichen Benchmark. Es gibt zur Zeit der Verfassung dieser Arbeit keine Möglichkeit nach deutschen Modellen zu filtern. Daher wird das aktuell generell beste mehrsprachige Modell verwendet. Aktuell ist das das "multilingual-e5-large-instruct"Modell, von intfloat, wer Teil von Microsoft Research in Asien ist. Dieses Model baut auf dem Model *lm-roberta-large"von FacebookAI auf, bevor es MetaAI wurde. Das Model wurde finetuned indem es mit 1 Milliarde Datensätzen kontrastiv vortrainiert wurde und im zweiten Schritt wurde es mit den Datensätzen, die für das E5-Mistral Model in der Arbeit Ïmproving Text Embeddings with Large Language Models"verwendet wurden, trainiert.

4.2 Auswahl der Vektor Datenbank

Als nächstes kommt die Auswahl der Vektordatenbank. Für diesen Protoyp wird eine Vektordatenbank verwendet, die in einem Docker Container betrieben wird. Im Bezug auf die Auswahl der Vektordatenbank werden drei Punkte beachtet.

1. Open Source
2. Selbst hostbar
3. Integration in Langchain

Hierfür stehen die zwei Optionen Chroma und Milvus zur Verfügung. Beide Optionen sind Open Source und selbst hostbar [**?**, **?**]. Beide Optionen erfüllen auch die zweite Anforderung und haben eine Langchain Integration [**?**]. Durch die Integration innerhalb von Langchain, gibt es für beide Datenbanken innerhalb der Entwicklung des Chatbots eine ähnliche Bedienung. Beide Optionen erfüllen die Anforderungen die für die Vektordatenbank erstellt wurden. Aufgrund von persönlicher Erfahrung mit ChromaDB wir aus diesen Gründen ChromaDB für diesen Prototyp ausgewählt.

4.3 Architektur des Systems

Nachdem die Auswahl des Development Tools und der Modelle abgeschlossen ist, ist die nächste Frage, welche Architektur das gesamte System haben soll. In **??** sieht man den Aufbau des Systems. Im folgenden wird die Umsetzung der spezifischen Teile des Systems besprochen.

4.3.1 Verbindungen

Der Chatbot wird auf einem REST-Server laufen, welcher auf einem dedicated Server läuft, der genauer in **??** erklärt wird. Der Rest Server ist in der prototypischen Implementierung ein Python Server, der FastAPI nutzt.

Die Python Applikation ist mit einer Vektordatenbank verbunden, die auf dem selben Server in einem Docker Container in Betrieb genommen wird. Die Daten die auf den Docker Container geladen werden kommen von der Confluence Instanz der Techdivision. Um die Daten auf die Datenbank zu laden gibt es ein weiteres Python System, welches die Daten von der REST-API des Confluence Systems abfragt und diese in die Datenbank überträgt.

Die Anbindung der Vektordatenbank und der Aufbau des RAG-Systems wird in **??** erklärt.

Das System wird erst ohne Anbindung an das externe Chat-System gebaut und wird daher erst mit Chainlit gebaut. Sobald hierbei alles abgeschlossen ist, wird die Verbindung zum anderen Chat-System hergestellt. Die Python Applikation des Chatbots ist dann mit der Google Chat API verbunden, welche darauf konfiguriert ist, über HTTPS-Server Instruktionen zu erhalten. Wie genau dies eingestellt ist und welche anderen Optionen es gibt wird in **??** erklärt.

4.3.2 Importer

Der Importer ist dafür zuständig, die Daten der jeweiligen Datenquellen in die Vektordatenbank zu laden. Hier ist es wichtig, dass die Datenquelle eine API hat von der die Einträge abgefragt werden können. Im Anschluss müssen die Einträge gereinigt werden und in ein Format und in eine Länge gebracht werden, mit der das Framework, die LLM und die Vektor Datenbank umgehen können.

4.3.3 Chatbot

Nach der groben Architektur des Systems kommt es zu der Architektur der einzelnen Teile. Begonnen wird hierbei mit dem Chatbot, also dem System, welches direkt auf dem Server laufen wird und das LLM hosten wird.

Framework

Bei der Auswahl der Konzeption für den Chatbot ist ein Frage, welches Framework verwendet werden soll. Die drei Frameworks die zu Auswahl stehen wurden in **??** vorgestellt. Haystack, Langchain und Llamaindex. Jedes der drei Frameworks ist nutzbar um einen nützlichen Chatbot mit RAG zu bauen. Bei allen drei gibt es die Möglichkeit mit RAG zu arbeiten und es ist ein Punkt auf den sich alle drei Stück fokussieren **??**.

Die Punkte in Hinsicht auf das Framwork, welches verwendet werden soll, die beachtet werden sind die Langlebigkeit des Projekts und ebenfalls die Menge an Dokumentation und

Support. Da die Welt der LLM Systeme sich aktuell schnell ändert und viele neue Erkentnisse in der Forschung gesammelt werden, verändern sich auch die AI Frameworks mit den LLM Systemen zusammen [**?**]. Da das System möglichst langlebig sein soll und im besten Fall nur die einzelnen Bauteile, wie beispielsweise die LLM, mit der Zeit ausgetauscht werden sollten, ist es wichtig zu beachten, dass es ein Framework ist, welches möglichst lange bestehen bleibt und das Austauschen von einzelnen Teilen ohne großen Aufwand erlaubt. Dies hilft auch bei der Entwicklung bzw. Weiterentwicklung, da dadurch verschiedene Konfigurationen ausprobiert werden können ohne große Veränderungen durchführen zu müssen. Die Community von Langchain ist die größte von den drei Framworks die in dieser Arbeit betrachtet werden, daher ist anzunehmen, dass der Support den man durch die Community erhält und die Menge an Dokumentation, die man finden kann hierfür am größten sind. Eine Github Suche nach Haystack gibt uns 1.7k Ergebnisse. Eine Suche nach Langchain gibt uns 25k Ergebnisse. Und eine Suche nach LlamaIndex gibt uns 1.3k Ergebnisse. Eine Google Trends Analyse in diesem Bereich wäre vermutlich nicht sehr aussagekräftig, da der Begriff Haystack von verschiedenen Unternehmen benutzt wird und dazu noch ein normal gebrauchtes Wort ist, daher werden wir uns an den Github Ergebnissen orientieren, da auch, wenn dies möglicherweise nicht alle Projekte auflistet, es uns zumindest eine Orientierungsrichtung gibt. Langchain hat hier den Vorteil.

Das Austauschen von einzelnen Teilen des Systems scheint mit allen Frameworks ähnlich aufwändig zu sein. Kleinere Änderungen können aufgrund der Chains in Langchain, der Pipelines in Llamaindex und der Pipelines in Haystack ähnlich leicht durchgeführt werden. Wenn man eine größere Änderung durchführen will, wie das Ändern von einem simplen Text-System zu einem multimodalen System, welches ebenfalls mit Bildern arbeiten kann dann würde es relevant aufwändige Änderungen in allen drei Frameworks erfordern. Daher wird hier angenommen, dass es bei allen drei Optionen den gleichen Aufwand kosten würde.

Hinzu kommt, dass Langchain und Llamaindex bereits Datenladesysteme für Confluence haben, welches das System ist welches an die Vektordatenbank angebunden werden soll. Dadurch, dass Langchain vielseitiger einsetzbar ist, als LlamaIndex und und Haystack, kommt der Vorteil dazu, dass Erfahrung mit dem Framework gesammelt werden kann und im Anschluss weitere Projekte, die über Chatbots und RAG Systeme hinaus gehen entwickelt werden können. Aus diesen Gründen wird Langchain für diese Projekt ausgewählt.

Nachrichten Historie

Der Chatbot hat standardmäßig keine Erinnerungen oder Kontext außerhalb der aktuellen Anfrage, der RAG Dokumente und des Prompts. Falls man in einer Nachricht auf den Kontext in einer vorherigen Nachricht eingehen möchte und diesen nicht erwähnt, dann weiß der Chatbot nicht worum es geht. Dies führt dazu, dass keine Folgefragen gestellt werden können, ohne dass der vorherige Kontext erklärt wird.

Um dieses Problem zu lösen muss eine Nachrichten Historie mit in das System eingebaut werden. Das Framework, Langchain, welches in **??** festgelegt wurde, gibt einem die Möglichkeit eine Nachrichtenhistorie einzubauen.

Den abgespeicherten Chats muss eine Session ID gegeben werden, damit die Chats abgerufen werden können.

Chats innerhalb des Spaces werden innerhalb von Threads geführt. Google gibt den Threads bereits eine ID. Da der Bot dieser Arbeit nur innerhalb dieses einen Saces arbeiten soll, wird diese ID verwendet um einen Python Hash Wert zu erzeugen, welche als Session ID verwendet wird. Diese Herangehensweise funktioniert nur innerhalb der Spaces, da die

API über HTTPS-Server nicht in der Lage ist, auf Threads innerhalb von Einzelchats zu antworten.

4.3.4 Server

Bei der Auswahl des Systems muss beachtet werden, dass die Techdivision den Server für weitere AI Projekte verwenden will und daher ein System, welches den Chatbot und die Datenbank betreiben kann ohne ausgelastet zu sein benötigt wird.

Das größte Modell, welches innerhalb dieser Arbeit verwendet wird, ist Phi-3 Medium von Microsoft, welches 14 Milliarden Parameter hat. Die Menge an VRAM, die ein Modell dieser Größe in voller Präzision benötigt ist 56GB VRAM. Die Standard Modelle, die von Ollama heruntergeladen werden, für diese Arbeit, sind 4 Bit quantisiert, wodurch die Anforderung auf 7GB VRAM sinkt. Die Uvicorn Worker, welche verwendet werden um den FastAPI Server als Service zu betreiben brauchen jeweils auch 2GB VRAM und RAM. Da der Server nicht alleine durch den Bot ausgelastet sein soll, wird nach einem Server mit einer VRAM Größe von mindestens 16GB gesucht. Dazu werden 64GB RAM benötigt. Hierfür stehen zwei Optionen zur Verfügung.

1. Eine Compute Engine Instanz in der Google Cloud
2. Ein dedicated Hetzner Server

Der Hetzner Server der zur Auswahl steht ist der aktuell einzige dedicated GPU Server, der bei Hetzner angeboten wird. Er hat einen Intel Core i5-13500 als CPU mit 6 Performance Cores und 8 Efficient Cores. 64 GB DDR4 RAM, 2x 1.92 TB Gen3 NVMe SSDs und eine Nvidia RTX 4000 SFF Ada Generation mit 20GB GDDR6 ECC. Die Kosten für diesen Server wären maximal, wenn der Server 730 Stunden im Monat läuft, 218.96€ + 94.01€ einmalige Setupgebühr.

Diese Hardware ist ausreichend für die Anforderungen des Systems und ermöglicht ihn für mehr Applikationen als diesen Chatbot zu benutzen.

Wenn man einen ähnlichen Server in der Compute Engine von Google zusammenstellen würde, dann hat man aktuell die Möglichkeit GPUs hinzuzufügen in den Maschinen Kategorien Allzweck, Rechen optimiert und Beschleunigungsoptimiert. Für die Berechnung der Kosten wird erneut angenommen, dass der Server 730 Stunden im Monat läuft. Wenn man einen N1 Allzweck Server mit 64GB RAM, 10 vCPUs, welche mindestens beötigt werden um 64 GB RAM auswählen zu können, und eine Nvidia T4 GPU mit 16GB VRAM hinzufügen, kostet diese Compute Engine Instanz 650€. Der Hetzner Server, ist bei voller Nutzung ca 430€ billiger im Monat und hat dazu 4GB mehr VRAM, daher wird der Hetzner Server ausgewählt.

4.3.5 Anbindung Google Chat

Die Anbindung an den Google Chat erfolgt über die Google Chat API. Hierbei gibt Google den Nutzern vier Optionen dafür, wie man mit der API interagieren kann.

Die vier Optionen sind:

1. App URL mittels https Server mit authentifiziertem SSL Zertifikat
2. Apps Script Projekt

3. Cloud Pub/Sub
4. Dialogflow

Dialogflow ist keine Option, da hier nicht die Option besteht eigene Open Source LLMs zu hosten und stattdessen die von Google angebotenen LLMs genutzt werden müssten, wodurch Anforderung A1 verletzt werden würde.

Apps Script ist auch keine Option, da bei Apps Scripts nur ein Skript ausgeführt wird, und keine vollwertige Umgebung zur Verfügung steht. Dies ist zu limitierend für diese Arbeit.

Da bereits ein HTTPS REST Server geplant ist, wird hier die App URL verwendet, wodurch direkt mit der API interagiert werden kann und die API Calls, bei ankommenden Nachrichten, direkt an den Server gesendet.

Pub/Sub wäre eine Alternative, falls eine Firewall im Weg stehen würde, was hier aber nicht der Fall sein wird, daher ist dieser Zwischenschritt nicht notwendig.

Für Google Chat gibt es die Möglichkeit Nachrichten synchron und asynchron zu versenden. Für eine synchrone Nachricht muss die Antwort innerhalb von dreißig Sekunden an die API gesendet werden, nachdem eine Nachricht erhalten wurde. Für den Nutzen innerhalb dieser Arbeit wäre dies genug, aber über synchrone Nachrichte, gibt es keine Möglichkeit auf Threads zu antworten, stattdessen werden einzelne Nachrichten versendet. Da die Threads benötigt werden, um einen sinnvollen Kontext für den Chatbot über die Nachrichtenhistorie zu erstellen, wird hier der asynchrone Ansatz gewählt.

5 Entwicklung

Nach dem Abschluss der Konzeption kann zur Entwicklung über gegangen werden. Die Rahmenbedingungen hierbei sind, dass der Hetzner Server, welcher in **??** beschrieben wurde verwendet wird und auf diesem Ubuntu 22 installiert ist.

5.1 Vorbereitung

Bevor mit der Entwicklung begonnen werden kann, müssen ein paar Vorbereitungen getroffen werden.

5.1.1 Modelle

Der erste Schritt ist es die Modelle vorzubereiten.

Um die Modelle vorbereiten zu können, wird als erstes Ollama installiert. Dies kann über das Install Skript, welches auf Github zu finden [**?**] ist getan werden. Dazu werden noch die Nvidia Cuda Driver installiert, damit Ollama diese nutzen kann. Nun können die Modelle vorbereitet werden.

Hierfür werden die Modelle, die in **??** vorgestellt wurden über Ollama gedownloadet. Die Modelle werden über `Ollama pull model` geladen, wobei `model` der Name des Models ist, welches man herunterladen möchte. Hierbei kann man mittels `model:latest` noch die Version angeben, die benötigt wird, wobei `latest`, die Version des Modells ist. Darüber kann man teils ebenfalls Größen angeben oder spezielle Versionen der Modelle Anfragen, wie

Instruct Modelle. Die verschiedenen Modelle haben verschiedene Optionen. Auf Ollama.com können die jeweiligen Modelle und die verschiedenen Versionen, eingesehen werden.

5.1.2 Google Cloud

Hier geht es um die Vorbereitung des Google Cloud Projekts für die Verbindung mit der Google Chat API.

Der erste Schritt hierbei ist es, ein Google Cloud Projekt zu erstellen. Wenn dies erledigt ist, muss die Google Chat API innerhalb des Projekts aktiviert werden.

Dazu wird ein Google Service Account benötigt, der auf die Google Chat API zugreifen kann. Die Konfiguration der API wird stattfinden, nachdem der Server konfiguriert wurde und das System funktionsfähig ist.

5.1.3 Datenbank

Hierfür wird auf dem Server Docker installiert. Dazu wird `chromadb/chroma` über `docker pull` heruntergeladen und dann mittels `docker run` gestartet.

5.2 System

Wie in der Konzeption beschrieben, ist das System auf dem der Chatbot erstellt wird ein Hetzner Server, auf dem ebenfalls die Datenbank liegt. Nachdem die Vorbereitungen des Servers abgeschlossen sind kann mit der eigentlichen Entwicklung begonnen werden.

Für die Entwicklung wird innerhalb dieser Arbeit erst Pseudo-Code gegeben und im Text wird dann die genaue Entwicklung mit den benutzten Frameworks beschrieben.

5.2.1 Importer

Für den Importer wird der Langchain `ConfluenceLoader` verwendet. Hierfür wird, nachdem der Docker Container, welcher die ChromaDB Datenbank beinhaltet gestartet wurde, eine Verbindung zu diesem hergestellt. Dabei wird dem Chroma Client innerhalb des Python Programms der `host` und der `port` des Docker Containers übergeben.

Danach wird das Embedding Modell mittels der `HuggingFaceEmbeddings` von langchain geladen. Hier wird der Name des Modells, ïntfloat/multilingual-e5-large-instruct"übergeben. Dazu gibt es die Option Keyword Arguments mittels `encode_kwargs` und `model_kwargs` mitzugeben. Hier geben wir das `device_kwargs` `device` mit dem Wert `cuda` mit. Dies sorgt dafür, dass die GPU für die Berechnungen verwendet wird, was den Prozess beschleunigt.

Nun können die Daten aus Confluence mittels `ConfluenceLoader` geladen werden. Hier werden `username`, `api_key` und die `url` der Confluence Instanz benötigt. Nach dem Erstellen des Loaders können die Daten geladen werden, hierfür gibt es die `load` Funktion, welche einen `space_key` braucht, für den Confluence Space der geladen werden soll.

Die geladenen Dokumente, werden mittels `MarkdownHeaderTextSplitter` ein mal aufgeteilt und mittels `RecursiveCharacterTextSplitter` noch ein weiteres mal aufgeteilt. Den aufgeteilten Dokumenten werden die Metadaten hinzugefügt, damit sie zugeordnet werden können.

Anschließend wird eine Langchain Chroma Instanz erstellt, die auf den vorher bereits erstellten Client zugreift, dann werden die Dateien mittels `from_documents` innerhalb einer Chroma Collection, die man beim Erstellen über die Chroma Instanz mittels `collection_name` festlegen kann, gespeichert.

5.2.2 Chatbot

Die Entwicklung des Chatbots wird in die einzelnen Teile der Langchain Chain aufgeteilt, die in **??** dargestellt ist. Die Chain besteht aus verschiedenen Element, die bereits zu Chains zusammengefügt wurden. Das Prompt und die LLM wurden mittels `create_stuff_documents_chain` miteinander verbunden. Diese Chain wurde dann mittels `create_retrieval_chain` mit einem Retriever verbunden. Als letztes wurde diese Chain in eine `RunnableWithMessageHistory` eingebaut. Im Folgenden werden die einzelnen Teile erklärt.

Retriever

Der erste Teil der Chain der entwickelt wird, ist der `Retriever`. Der Retriever ist der Teil der Chain, der auf die Vektordatenbank zugreift und die passenden Dokumente von dieser an die LLM übergibt.

Hierfür wird, wie beim Importer in **??** ein Chroma Client erstellt, der mit dem Docker Container der Chroma Instanz verbunden wird, dazu wird ebenfalls synchron zum Importer, das Embedding Modell über `HuggingFaceEmbeddings` geladen und in der Chroma Instanz als `embedding_function` angegeben. Dieser wird dann mit der Funktion `as_retriever()` als Retriever geladen.

Für die Chain wird dazu noch eine Funktion erstellt, welche die Dokumente, die der Retriever abruft zusammenfügt, damit diese der LLM übergeben werden können.

Prompt

Das Prompt wird mittels `ChatPromptTemplate.from_messages` erstellt. Das Prompt, welches benutzt wird, ist Folgendes:

Dein Name ist Arthur. Du bist ein erfahrenes Mitglied des IT-Support-Teams der Firma TechDivision. Deine Aufgabe ist es, Mitarbeitern bei ihren IT-Problemen zu helfen. Beachte folgende Richtlinien:

1. Nutze ausschließlich die Informationen aus dem bereitgestellten Kontext und deinem vorhandenen Wissen über allgemeine IT-Konzepte, um Fragen zu beantworten.
2. Wenn die Antwort nicht im Kontext oder in deinem allgemeinen IT-Wissen enthalten ist, sage ehrlich, dass du die Information nicht hast oder die Frage nicht beantworten kannst.
3. Erfinde oder halluziniere keine Informationen.
4. Beziehe dich auf frühere Teile der Konversation, wenn sie relevant sind.
5. Bleibe stets höflich, professionell und hilfsbereit.
6. Wenn nötig, frage nach weiteren Details, um das Problem besser zu verstehen.

7. Gib schrittweise Anleitungen, wenn du eine Lösung vorschlägst.
8. Verweise den Mitarbeiter an andere Abteilungen oder Experten, wenn das Problem außerhalb deines Zuständigkeitsbereichs liegt.
9. Antworte auf Deutsch.

Kontext: {`context`} Bisheriger Chatverlauf: {`chat_history`} Beantworte die folgende Frage des Mitarbeiters basierend auf diesen Informationen.

Wobei context, über über die Chain übergebene RAG Dokumente sind und chat_history ebenfalls über die Chain übergebene Chat Historie ist. Die Richtlinien, sollen der LLM eine Richtung geben und dazu falsche Informationen und Halluzinationen vermeiden.

LLM

Dieser Schritt kann erst funktionieren, wenn die in **??** beschriebenen Vorbereitungen abgeschlossen sind. Die LLM wird dafür mit der `Ollama` Community LLM geladen. Wobei der Klasse der Model name aus Ollama übergeben wird.

Nachrichten Historie

Die Nachrichten Historie wird mittels `RunnableWithMessageHistory` implementiert. Hierfür wurde noch eine `get_session_history` Funktion implementiert, welche aus einer SQLite Datenbank mittels der `session_id` den aktuellen Session Chat abruft. Dem `RunnableWithMessageHistory` muss ein Runnable Objekt übergeben werden, welches als Basis dient. Hier ist das Basis Runnable Objekt, die Retrieval Chain, die aus dem Prompt, dem Retriever und der LLM besteht. Dazu wird der Runnable die `get_session_history` Funktion übergeben. Da das Basis Runnable ein dict als Input benötigt, muss ein `input_messages_key` angegeben werden, was laut dem Prompt, welches verwendet wird der Key `input` ist. Dazu wird der `output_messages_key` `answer` übergeben und der `history_messages_key` `chat_history`.

Dieses `RunnableWithMessageHistory` Runnable kann mit `runnable.invoke()` angesprochen werden. Der Funktion werden zwei dict Objekte mitgegeben. Das dict mit dem `input` Key, der die Eingabe des Nutzers ist und dazu noch ein weiteres dict mit einem `configurable` Key, dessen Wert ein dict ist, mit einem `session_id` key.

5.2.3 Google Cloud API

Um async Nachrichten an die Google Chat API zu senden, wird eine Credential Datei benötigt, von einem Service Account, der mit der Google Chat API verbunden ist. Im Code wird der Scope `https://www.googleapis.com/auth/chat.bot` spezifiziert und aus dem Scope und dem Service account wird mit der `build` Funktion der client erstellt. Die Nachricht mit der Funktion `Client.spaces().message().create().execute()` erstellt und abgesendet. Die Funktion bekommt die notwendigen Informationen für die Nachrichten übergeben. Aus diesen wird die Nachricht aufgebaut und mit der `messageReplyOption` `REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD` abgesendet. Diese sorgt dafür, dass, falls ein Thread, der existiert, angegeben ist, im jeweiligen Thread geantwortet wird und sonst wird außerhalb des Threads geantwortet.

5.2.4 Nachrichtenverarbeitung

Für die Verbindung an die Google Cloud API wird ein POST Endpunkt erstellt. Dieser Endpunkt nimmt die Nachrichten, die in Google Chat an den Chatbot gesendet werden an. Die JSON Nachricht, die man von der API erhält beinhaltet viele Informationen. Für den Chatbot sind die ID des Nutzers, der die Nachricht abschickt, der Thread in dem die Nachricht versendet wurde und die Nachricht, die abgesendet wurde, wichtig. Dazu wird noch die Email überprüft, die die Nachricht abgeschickt hat.

Zuerst wird überprüft, ob der Email Host ein valider Email Host ist, der Zugang auf diesen Bot haben sollte.

Die ID des Nutzers, die unter `message.sender.name` zu finden ist, wird benutzt um den Nutzer in der Nachricht anzusprechen. Die Nachricht, die abgesendet wird beginngt hierfür mit @message.sender.name und dadurch kann über die API ein Nutzer angesprochen werden. Wenn man den `message.sender.displayName` verwendet, dann wird der Nutzer nicht in der Nachricht verlinkt, sondern es ist nur ein Text, der im Google Chat ankommt.

Der Thread ist unter `message.thread.name` zu finden. Die Thread ID, welche der letzte Teil der Nachricht ist, wird verwendet um die Session ID herzustellen und sie wird als `thread.name` an die API geschickt.

Ein GoogleChatClient wird initialisiert und die notwendigen Infos werden an diesen übergeben und die Nachricht wird abgesendet.

5.2.5 Server Konfiguration

Damit die Google Chat API den Server akzeptiert, wird ein SSL Zertifikat benötigt. In diesem Fall wurde Nginx verwendet um das SSL Zertifikat zu laden und als Reverse-Proxy für den FastAPI Server. Der FastAPI Server wird als Service gestartet und für die Demo wurden 4 Unicorn Worker, für den FastAPI Server benutzt.

6 Vergleich

6.1 Evaluierung

Der finale Teil der Arbeit beschäftigt sich mit dem Vergleich der Performance, des Systems. Das System wurde mit vier verschiedenen LLM Modellen implementiert und wird im ersten Schritt mit dem Framework RAGAS (siehe **??**) miteinander verglichen. Nach diesem Schritt werden die zwei bestabschneidensten Modelle für die finale Bewertung von Mitarbeitern aus der Techdivision miteinander verglichen. Im Konzept wurden die meisten Anforderungen aus **??** bereits erfüllt. Hier werden **??** und **??** erneut aufgegriffen.

Modelle

Die vier LLM Modelle die bewertet werden sind Metas Llama3, Mistrals Mistral 2, Googles Gemma2 und Microsofts Phi-3.

Der größte Unterschied den man zwischen diesen Modellen sieht ist die Parametergröße der Modelle. Mistral hat hierbei 7 Milliarden Parameter, Llama3 hat 8B, Gemma2 hat 9B und Phi-3 ist das größte Modell mit 14B Parametern.

Alle Modelle sind Decoder Only Transformer, wobei Mistral sliding window attention nutzt, Gemma2 variiert zwischen sliding window und global attention, Phi-3 nutzt blocksparse attention. Dazu nutzen alle Modelle Grouped-query Attention. Gemma 2 wurde mit 8 Billionen Tokens trainiert, Llama3 mit 15 Billionen und Phi-3 mit 4.8 Billionen **??**. Die Menge von Tokens für Mistral ist nicht bekannt.

6.2 RAGAS Vergleich

Ragas ist das framework, welches zum bewerten und vergleichen der LLMs innerhalb dieser Arbeit verwendet wird. Das Framework dient dazu RAG pipelines zu bewerten. Im Rahmen davon lassen sich Testsets generieren, die Fragen, Antworten und die dazu passenden RAG Dokumente beinhalten.

Über diese Testsets lassen sich die Pipelines über verschiedene Metriken vergleichen. Die Metriken die zur Verfügung stehen und die die im finalen Vergleich relevant sind, werden in **??** genauer beschrieben.

Die erste Phase des Evaluierungsprozesses findet über dieses Framework statt.

6.2.1 Testset

Der erste Schritt für den Vergleich ist es Testsets herzustellen anhand derer Ragas den Benchmark Test durchführen kann.

Die Variante die hierfür verwendet wird ist es einen TestsetGenerator per `TestsetGenerator.from_langchain()` herzustellen und mit diesem dann das Testset erstellen zu lassen.

Als Vorbereitung hierfür werden Daten benötigt. Die Daten die als Dokument geladen werden haben alle ein Metadata Dictionary. Damit der Generator erkennen kann, welche Daten zusammengehören brauchen die Dokumente in diesem Metadata Dictionary ein Key Value Pair mit dem Namen ”filename“. Der Wert des ”filename“ Schlüssels wird verwendet um bei der Generation der Testdaten zusammengehörige Dokumententeile zusammenzufügen.

Nach dem Vorbereiten der Daten kann man damit beginnen den Generator zu erstellen. Für diesen Generator werden drei Parameter verwendet. Die `generator_llm`, `critic_llm` und `embeddings`. Als Embedding Model benutzen wir ebenfalls das Model ïntfloat/multilingual-e5-large", wie für den Chatbot und den Importer. Als Critic und als Generator LLM wird das 70 Milliarden Parameter Model Llama3 von Meta verwendet. Die Nutzung von Hugginface Modellen mit Ragas scheint aktuell nicht zu funktionieren, daher wird das Llama3 Model, welches man direkt über Ollama beziehen kann, verwendet. Nach dem Erstellen des Generators und dem vor und aufbereiten der Daten kann man das Testset generieren lassen über die Funktion `generate_with_langchain_docs()`. Für die Generierung kann man drei Parameter setzen. Als erstes die `documents` von denen man die Testsets generieren will. Hier werden Documents verwendet die über den `ConfluenceLoader` von Langchain geladen werden. Parallel wie beim Importer (**??**). Dazu kann man bestimmen, wie viele Elemente im Set enthalten sein sollen. Diese `test_size` bestimmt wie viele Fragen vom `TestsetGenerator` generiert werden. Dazu kann man die Verteilung der Fragen bestimmen. Die Fragen die durch den Generator erstellt werden

können dazu drei verschiedene Typen haben. Diese drei Typen sind: `simple`, `reasoning` und `multi_context`. Diese kann man der `generate` Funktion als dict mitgeben mit der Verteilung die man verwenden möchte. Hier werden diese Werte verwendet: `simple: 0.5`, `reasoning: 0.2` und `multi_context: 0.3`.

Das Testset welches hierbei generiert wird kann man mittels `to_pandas()` als pandas Dataframe ausgeben. Wenn man dieses Dataframe betrachtet sieht man, 4 relevante Spalten.

`question`: Die Frage die dem Chatbot für das Benchmarking gestellt wird.

`context`: Der Context aus dem die Frage erstellt wurde.

`ground_truth`: Die Antwort auf die Frage.

`question_type`: Der Question Typ, dessen Verhältnis in der `generate` Funktion bestimmt wurde.

Mit diesem Testset kann man nun die Chain bewerten.

6.2.2 Evaluation

Nachdem das Testset generiert wurde kann mit der Bewertung begonnen werden. Für die Bewertung gibt es acht verschiedene Metriken, die man betrachten kann. Hiermit kann man die verschiedenen Teile der Rag-Pipeline bewerten. Im Folgenden werden die acht Metriken die zu Verfügung stehen betrachtet. Standardmäßig verwendet die `evaluate` Funktion die OpenAI API, dies kann überschrieben werden indem man der der Funktion ein `llm` und ein `embedding` übergibt. Man kann hierbei die Llm und das Embedding Model in `LangchainLLMWrapper` und in `LangchainEmbeddingsWrapper` verpacken, falls man kein Langchain LLM oder Embedding verwendet. Wenn die LLM und das Embedding von Langchain ist, dann kann man diese direkt übergeben und sie werden in der Funktion verpackt.

Faithfulness

"Faithfulness" zu Deutsch "Genauigkeit" ist die Metrik, die die generierte Antwort mit dem abgerufenen Kontext vergleicht. Dieser Wert ist eine Gleitkommazahl zwischen 0 und 1. Hierbei werden erst die Behauptungen aus der Antwort heraus gesucht und diese werden dann mit dem abgerufenen Kontext verglichen und es wird überprüft, ob die Behauptung aus dem Kontext gezogen werden kann.

Die Formel die hierfür verwendet wird ist wie folgt:

$$\text{Faithfulness score} = \frac{|\text{Anzahl der Behauptung die man aus dem Kontext schließen kann}|}{|\text{Gesamtzahl der Behauptungen}|}$$

Answer relevancy

"Answer relevancy" zu Deutsch "Relevanz der Antwort", bewertet wie relevant die Antwort für die Eingabe ist. Hier werden Antworten mit überflüssigen Informationen oder unvollständige Antworten geringer bewertet und Antworten die die Frage vollständig beantworten und keine überflüssigen Informationen beinhalten, werden höher bewertet. Der Wert der Relevanz der Antwort kann zwischen -1 und 1, ist in der Praxis aber meisten zwischen 0 und 1 sein. Die Formel die hierfür verwendet wird ist:

$$\text{answer relevancy} = \frac{1}{N}\sum_{1}^{N}\cos(E_i, E_0)$$

Model Bewertung				
Metrik	llama3	mistral2	phi3	gemma2
answer_relevancy	0.9998	0.9802	0.9981	0.7481
answer_correctness	0.6126	0.3717	0.4172	0.5011
answer_similarity	0.9032	0.8935	0.8992	0.8712
faithfulness	0.7415	0.8444	0.6852	0.5938

Abbildung 6.1 Ragas Ergebnisse der vier Modelle.

Answer semantic similarity

Bei der Metrik "Answer semantic similarity", zu Deutsch "Semantische Ähnlichkeit der Antwort", werden die `answer` und `ground_truth` miteinander verglichen. Hierbei wird die Semantische Ähnlichkeit der beiden Werte gemessen. Dieser Wert ist eine Gleitkommazahl zwischen 0 und 1. Ein höherer Wert steht hierbei für eine höhere Genauigkeit. Hierfür wird das embedding Model verwendet um die `answer` und `ground_truth` in Vektoren einzubetten und diese werden dann mittels Kosinus-Ähnlichkeit verglichen. Die Formel hierfür ist in **??** bereits vorgestellt worden.

Answer correctness

Bei der Metrik "Answer correctness" oder zu Deutsch "Korrektheit der Antwort", wird die Korrektheit der Antwort überprüft. Dieser Wert ist ebenfalls eine Gleitkommazahl zwischen 0 und 1. Höhere Werte bedeuten hier eine höhere Korrektheit. Hierfür werden `ground_truth` und `answer` miteinander verglichen. Hierfür wird einmal die Semantische Ähnlichkeit betrachtet und dazu die faktische Ähnlichkeit. Diese Werte werden im Anschluss mittels einer gewichtet Formel benutzt um den Answer correctness Score zu erhalten.

$$\text{F1 Score} = \frac{|\text{TP}|}{(|\text{TP}| + 0.5 * (|\text{FP}| + |\text{FN}|))}$$

Wobei TP Richtig positive Fakten sind, Werte die laut Antwort wahr sind und auch in der `ground_truth` enthalten sind. FP sind falsch positive Fakten. Fakten die laut `answer` wahr sind aber nicht in der `ground_truth` enthalten sind. FN sind falsch negative Fakten. Fakten die in der `ground_truth` enthalten sind, aber nicht in der `answer`.

6.2.3 Ergebnisse

Nach erstellen von mehreren Testsets mit verschiedene Modellen sind bei einer `test_size` von 10 jedes mal mehrere Reihen in der `ground_truth` Spalte leer geblieben. Deshalb wurde ein Testset verwendet, welches 8 Reihen hatte, bei denen alle Werte gefüllt waren.

Bei den Metriken, die für diesen Vergleich in Betracht gezogen worden sind, wurden die Metriken, die nur mit der RAG Pipeline zu tun haben entfernt, da diese den selben Wert für alle haben. Es wurden nur die Metriken betrachtet, auf die die genutzte LLM einen Einfluss hat.

In der Tabelle **??** sieht man die Ergebnisse des Ragas Benchmarks. Die verschiedenen Metriken die betrachtet wurden sind `answer_relevancy`, `answer_correctness`,

`answer_similarity`, `context_relevancy` und `faithfulness`. Alle Metriken außer `faithfulness` sind am höchsten bei Llama3 von Meta. `Faithfulness` hat den höchsten Wert bei Mistral 2 von Mistral. Das beste Modell hierbei ist Llama3 mit 3 Metriken bei denen es am besten bewertet wurde. Das zweitbeste ist Phi-3 mit zwei Metriken als zweitbestes Modell und zwei als drittes. Das drittbeste Modell ist Mistral mit einer Bewertung als bestes Modell, zwei als drittes und eins als viertes. Das schlechtest Modell innerhalb dieser Ragas Bewertung ist Gemma2 mit einer Metrik als zweitbestes und drei als viertes.

Das worin sich Llama3 von den anderen Modellen am meisten unterscheidet ist die Menge an Tokens die zum trainieren verwendet wurden. Mit 15 Billionen Token ist dies 7 Billionen von den für Gemma 2 verwendet 8 Billionen Token entfernt. Aber das zweitbeste Modell in diesem Vergleich, Phi-3, hatte die kleinste bekannte Menge an Trainingsdaten, aber dafür war es mit 14 Milliarden Parametern das größte Modell. Aufgrund dessen lässt sich nicht direkt ein eindeutiger Grund sehen, wieso hier einige Modelle besser als die anderen abschneiden.

6.3 Umfrage Vergleich

Dieser Schritt der Bewertung greift Anforderungen A4 und A7 erneut auf. A4 wurde bereits durch die RAG Pipeline und den Importer in den Chatbot eingebaut, der nächste Schritt ist es zu überprüfen, ob die richtigen Unternehmensdaten verwendet werden und diese logisch und schlüssig in die Antworten mit eingebaut werden. Hierfür wurde 4 Mitarbeitern aus dem Unternehmen ein Fragebogen zum Ausfüllen gegeben und dazu wurde ihnen Zugang zum System freigeschaltet. Dies bewertet gleichzeitig die Antworten und ermöglicht es die 2 Modelle miteinander zu vergleichen.

6.3.1 Fragebogen

Der Fragebogen wurde so erstellt, dass es keine genauen Fragen gibt, sondern nur Themengebiete, die abgefragt werden sollen. Das wichtige beim Auswählen der Themengebiete war, dass es Informationen waren, die auf der Confluence Instanz, auf die der Chatbot Zugriff hatte, verfügbar waren. Die Confluence Instanz die für diese Arbeit verwendet wurde ist ein Testsystem auf das für diese Arbeit verschiedene Blogeinträge und Dokumentationen geladen wurden, die den Einträgen auf dem Produktivsystem ähneln nur ohne kritische Kundendaten und ähnliches. Die Fragen für den Fragebogen werden aus diesen neu erstellten Einträgen erstellt.

Folgende Aufgaben wurden den Teilnehmern gegeben:

1. Wie richten sie die VPN ein? Lassen sie sich eine Anleitung dazu ausgeben.
2. Wie lösen Sie bei aktiven VPN zur TD das Problem von DNS Auflösung?
3. Welche Möglichkeiten gibt es zum Teilen von sensitiven Daten?

Die den Teilnehmern gegeben Aufgaben decken unterschiedliche Kompetenzen des Chatbots ab.

Frage 1, die Einrichtung der VPN: Der Chatbot sollte in der Lage sein, eine klare, schrittweise Anleitung zur VPN-Einrichtung aus dem Confluence-Wissen zu extrahieren und dem Teilnehmer präsentieren.

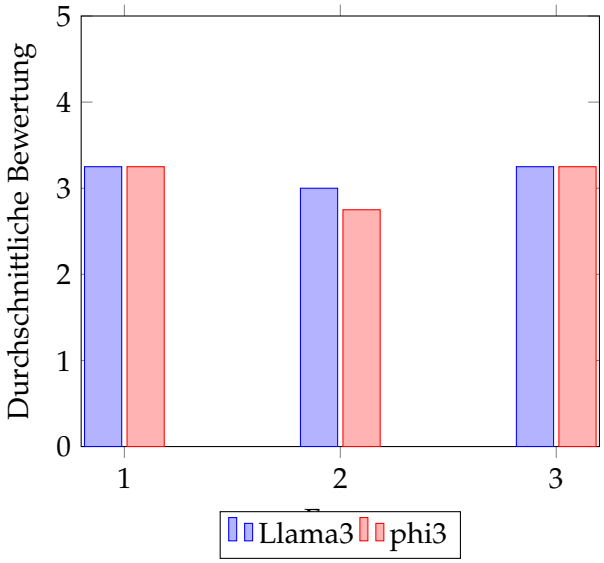


Abbildung 6.2 Bewertungen von Llama3 und Phi3

Frage 2, DNS-Auflösungsproblem bei aktivem VPN: Hier sollte der Chatbot Problemlösungsstrategien aus dem Confluence ableiten und dem Benutzer mögliche Lösungsansätze vorschlagen.

Frage 3, Teilen sensitiver Daten: Der Chatbot sollte verschiedene in Confluence dokumentierte Methoden zum sicheren Datenaustausch identifizieren und dem Benutzer erklären können.

Die Teilnehmer sollten die Antworten auf einer Skala von 1-5 für beide LLMs bewerten. wobei 1 für sehr schlecht und 5 für sehr gut steht.

In **??** sind die durchschnittlichen Bewertungen der LLMs dargestellt. Frage 1 und Frage 3 haben den selben durchschnittlichen Wert bei beiden LLMs wie in **??** erkennbar. Frage 2 ist die Frage in der der durchschnittliche Wert unterschiedlich ist. Hier ist der durchschnittliche Wert bei Llama3 um 0.25 besser.

In Worten sind beide LLMs bei der Frage in der eine Anleitung erstellt werden sollte und der Frage ob über die Möglichkeiten zum sicheren Datenaustausch etwas besser als Okay. Bei der Frage in der die LLMs dem Nutzer Problemlöstungsstrategien geben sollten, ist Llama3 Okay und Phi-3 etwas schlechter als Okay.

Dies führt dazu, dass Llama3 insgesamt die bessere Bewertung hat und somit im Vergleich besser ist. Genau wie in **??** bereits erwähnt, ist der größte Unterschied zwischen Llama3 und den anderen Modellen die Menge an für das Trainieren verwendeten Daten. Da Phi-3 besser als Gemma 2 im Ragas Vergleich abgeschlossen hat lässt sich dies aber zumindest in dem Test widerlegen.

7 Fazit

Die Ergebnisse die in dieser Bachelorarbeit erzielt wurden, bieten nicht nur wichtige Erkenntnisse für die Entwicklung und für den Einsatz fortschrittlicher Chatbot-Systeme. Die gewonnenen Ergebnisse haben nicht nur akademischen Wert, sondern können auch als Grundlage für zukünftige Entwicklungen und Implementierungen dienen.

7.1 Zusammenfassung Konzeption und Entwicklung

Die Konzeption des Chatbots hat gezeigt, dass es bereits heute verschiedene Möglichkeiten, für die Entwicklung eines Chatbots, gibt. Jeder Teil des Systems von der Datenbank, bis zum Framwork, hat einige verschiedene Alternativen die alle ihre eigenen Vorteile bieten.

In dieser Arbeit wurde ein Konzept für einen RAG-Chatbot entwickelt und dieses wurde prototypisch mit dem Framework Langchain entwickelt und in Google Chat eingebaut. Dieser Übergang von theoretischem Konzept und prototypischer Implementierung hat das Ziel der Überbrückung von Theorie und Praxis der Arbeit erfüllt. Die Entwicklung zeigte, dass moderne Frameworks und Tools die Erstellung KI-gestützter Chatbots erheblich erleichtern, wobei dennoch eine sorgfältige Auswahl und Integration der Komponenten erforderlich ist, um ein optimales Ergebnis zu erreichen. Die Entwicklung des Chatbots hat das Ziel der praxisorientierten Lösung für ein Chatbot System innerhalb dieser Arbeit erfüllt.

7.2 Zusammenfassung Bewertung

Im Rahmen der Bewertung wurden vier verschiedene LLMs miteinander verglichen, wobei ein zweistufiger Evaluierungsprozess angewandt wurde. Im ersten Schritt wurde ein Vergleich über das Framework RAGAS durchgeführt, welches speziell für die Bewertung von RAG-Systemen entwickelt wurde. RAGAS ermöglichte eine objektive Bewertung verschiedener Aspekte der Modellleistung, einschließlich der Genauigkeit, Relevanz und Konsistenz der generierten Antworten.

Im zweiten Schritt wurde ein Vergleich mittels eines Fragebogens durchgeführt. Dieser qualitative Ansatz erlaubte es, die Nutzererfahrung und die praktische Anwendbarkeit der verschiedenen Modelle zu bewerten. Die Kombination aus quantitativer (RAGAS) und qualitativer (Fragebogen) Evaluation lieferte ein umfassendes Bild der Stärken und Schwächen der untersuchten LLMs.

Die Ergebnisse zeigten, dass LLaMA 3 in beiden Bewertungsphasen am besten Abschnitt. Dies unterstreicht das Potenzial von Open-Source-Modellen in der Entwicklung leistungsfähiger Chatbots. Dies erfüllte das Ziel des Herausfindens der besten LLM der vier Optionen für diesen spezifischen Anwendungsfall.

7.3 Ausblick

Die Erkentnisse dieser Arbeit eröffnen mehrere Richtungen für zukünftige Forschung und Entwicklung.

Die erfolgreiche Implementation des RAG-Chatbots zeigt das Potenzial dieser Technologie. In der Bewertung mit Ragas hat sich gezeigt, dass es noch Potenzial für Verbesserung gibt.